

# **Project Report**

## **STUDENT ISSUE TRACKER**

### **Submitted By:**

BSCS-E-22-05

Affifa Fatima

2022 – 2026

### **Supervised By:**

Dr. Aamir Raza



**DEPARTMENT OF COMPUTER SCIENCE**  
**BAHAUDDIN ZAKARIYA UNIVERSITY MULTAN**  
**PAKISTAN**

## FINAL APPROVAL

This is to certify that we have read this report titled “**Student Issue Tracker**” submitted by **Affifa Fatima BSCS-E-22-05** and it is our judgment that this report is of sufficient standard to warrant its acceptance by Bahauddin Zakariya University, Multan for the degree of BS (Computer Science).

### Committee:

#### 1. External Examiner

---

<<Examiner Name>>

<<Designation>>

<<Organization>>

#### 2. Supervisor

---

Dr. Aamir Raza

Assistant Professor,

Department of Computer Science

Bahauddin Zakariya University, Multan

#### 3. Chairperson

---

Dr. Humaira Afzal

Professor

Department of Computer Science

Bahauddin Zakariya University, Multan.

## DECLARATION

I hereby declare that this thesis report titled **Student Issue Tracker** is based on my own project work carried out for the Bachelor of Science in Computer Science program. The system has been designed and implemented as a web-based complaint management platform for university students and administrative staff. All external tools, frameworks, libraries, and references used in the project have been acknowledged in the relevant sections of this report.

Signature:

---

Date:

---

## DEDICATION

*I dedicate this project to my parents for their unconditional love, sacrifices, and constant encouragement throughout every stage of my life. I am deeply grateful to my teachers and mentors for their guidance, support, and valuable knowledge throughout my academic journey.*

## ACKNOWLEDGMENT

All praises is due to Allah Almighty, the Most Gracious and Most Merciful, who blessed me with the strength, patience, and ability to complete this Final Year Project.

I express my sincere gratitude to my supervisor **Dr. Aamir Raza** for guidance, technical feedback, and continuous support throughout the analysis, design, development, testing, and documentation phases of this project.

I am also thankful to the Department of Computer Science, Bahauddin Zakariya University, Multan, for providing an academic environment that encouraged practical learning and problem solving. I appreciate the help and encouragement of my family, friends, and classmates during the completion of this work.

*Affifa Fatima*

# PROJECT BRIEF

|                  |                                                                                                                                                                                                                                             |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Title    | Student Issue Tracker: University Complaint Management System                                                                                                                                                                               |
| Domain           | Software Engineering / Full-Stack Web Development / Academic Administration                                                                                                                                                                 |
| Supervisor       | <b>Dr. Aamir Raza</b> Department of Computer Science, Bahauddin Zakariya University (BZU), Multan                                                                                                                                           |
| Session          | 2022 – 2026                                                                                                                                                                                                                                 |
| Technology Stack | Next.js App Router, React, Next.js API Routes, Supabase Auth, Supabase PostgreSQL, Supabase Storage, Gmail SMTP, Botpress Cloud Webchat, Browser Speech Synthesis, Custom SVG Analytics, JavaScript AI Utilities, and Lucide-React UI Icons |
| Undertaken By    | Affifa Fatima<br>BSCS-E-22-05                                                                                                                                                                                                               |
| Starting Date    | 5th October, 2025                                                                                                                                                                                                                           |
| Completion Date  | 11th May, 2026                                                                                                                                                                                                                              |

# ABSTRACT

The Student Issue Tracker is a full-stack web-based complaint management system developed for university departments. It provides a centralized platform where students can register, submit complaints, receive AI-assisted category and priority suggestions, upload supporting attachments, track complaint progress, search complaint records, view details through modal dialogs, and provide feedback after resolution.

The system replaces informal complaint handling methods with a structured workflow that improves transparency, accountability, and administrative efficiency. The project supports five major roles: Student, Faculty Member, HOD, DSA, and Supervisor. Complaints are routed by category: Academic and Behavior-related complaints go to HOD, while Administrative, Facilities, and Other complaints go to DSA. HOD and DSA users review complaints, accept or reject them, and assign accepted work to Faculty Members.

Faculty Members view assigned complaints, contact students when required, upload evidence, and mark work as resolved. DSA or Supervisor users finalize resolved complaints, after which students can submit ratings and feedback.

The implemented system uses Next.js, React, Next.js API Routes, Supabase Auth, Supabase PostgreSQL, Supabase Storage, Gmail SMTP, Botpress, custom SVG analytics, browser speech synthesis, responsive CSS, and JavaScript AI utilities. It also includes profile editing, role-specific dashboard summaries, searchable complaint queues, modal-only complaint descriptions, draggable chatbot and voice-assistant widgets, notifications, activity logs, analytics, weekly reports, department management, staff management, and attachment support.

## TABLE OF CONTENTS

|                            |            |
|----------------------------|------------|
| <b>DECLARATION.....</b>    | <b>i</b>   |
| <b>DEDICATION.....</b>     | <b>ii</b>  |
| <b>ACKNOWLEDGMENT.....</b> | <b>iii</b> |
| <b>PROJECT BRIEF.....</b>  | <b>iv</b>  |
| <b>ABSTRACT.....</b>       | <b>v</b>   |

### CHAPTER 1

#### INTRODUCTION 1

|                                        |   |
|----------------------------------------|---|
| 1.1 Background and Motivation .....    | 2 |
| 1.2 Problem Statement.....             | 3 |
| 1.3 Project Objectives .....           | 4 |
| 1.4 Scope of the Project.....          | 4 |
| 1.5 Significance of the Project.....   | 5 |
| 1.6 Methodology Overview .....         | 6 |
| 1.7 Limitations .....                  | 6 |
| 1.8 System Architecture Overview ..... | 7 |

### CHAPTER 2

#### SYSTEM ANALYSIS 9

|                                            |    |
|--------------------------------------------|----|
| 2.1 Feasibility Study .....                | 10 |
| 2.1.1 Technical Feasibility .....          | 10 |
| 2.1.2 Operational Feasibility .....        | 11 |
| 2.1.3 Economic Feasibility.....            | 11 |
| 2.1.4 Schedule Feasibility .....           | 11 |
| 2.1.5 Legal and Security Feasibility ..... | 12 |
| 2.2 Data Gathering Techniques.....         | 12 |
| 2.2.1 Questionnaires .....                 | 12 |
| 2.2.2 Sampling and Observations.....       | 13 |



|                                                         |    |
|---------------------------------------------------------|----|
| 2.2.3 Interviews .....                                  | 13 |
| 2.3 Functional Requirements .....                       | 13 |
| 2.4 Non-Functional Requirements .....                   | 14 |
| 2.5 Data Flow Diagram - Level 0 (Context Diagram) ..... | 15 |
| 2.6 Data Flow Diagram - Level 1 .....                   | 15 |

## **CHAPTER 3**

|                                                                        |    |
|------------------------------------------------------------------------|----|
| <b>SYSTEM DESIGN</b> .....                                             | 17 |
| 3.1 Proposed System and Its Features.....                              | 18 |
| 3.2 Use Case Diagram .....                                             | 18 |
| 3.3 Sequence Diagrams .....                                            | 19 |
| 3.3.1 Sequence Diagram – User Login.....                               | 20 |
| 3.3.2 Sequence Diagram – Complaint Submission with AI Assistance ..... | 20 |
| 3.3.3 Sequence Diagram – Review and Faculty Assignment .....           | 21 |
| 3.3.4 Sequence Diagram – Resolution and Finalization.....              | 22 |
| 3.4 Entity Relationship Diagram (ERD) .....                            | 23 |
| 3.5 Relational Schema .....                                            | 24 |
| 3.6 Normalization to 3NF .....                                         | 25 |

## **CHAPTER 4**

|                                                                      |    |
|----------------------------------------------------------------------|----|
| <b>SYSTEM DEVELOPMENT AND IMPLEMENTATION</b> .....                   | 27 |
| 4.1 Technology Stack .....                                           | 28 |
| 4.2 Project Structure .....                                          | 29 |
| 4.3 Authentication and RBAC Implementation .....                     | 29 |
| 4.4 Complaint Workflow Implementation .....                          | 30 |
| 4.5 AI Assistance Implementation .....                               | 30 |
| 4.6 Notification and Email Implementation.....                       | 31 |
| 4.7 Attachment Implementation.....                                   | 32 |
| 4.8 Analytics .....                                                  | 32 |
| 4.9 Botpress Chatbot Integration .....                               | 32 |
| 4.10 Voice Assistant, Search, and Responsive UI Implementation ..... | 33 |

## **CHAPTER 5**

|                             |    |
|-----------------------------|----|
| <b>USER GUIDE</b> .....     | 35 |
| 5.1 Getting Started .....   | 36 |
| 5.2 Login Page .....        | 36 |
| 5.3 Registration Page ..... | 37 |
| 5.4 Student Dashboard.....  | 38 |

|                                        |    |
|----------------------------------------|----|
| 5.6 AI Suggestions Panel.....          | 38 |
| 5.7 Complaint Tracking Interface ..... | 39 |
| 5.8 HOD Dashboard .....                | 39 |
| 5.8.1 HOD Analytics Dashboard.....     | 40 |
| 5.8.2 Complaint Review Modal .....     | 40 |
| 5.9 DSA Dashboard .....                | 41 |
| 5.9.1 Complaint Section.....           | 41 |
| 5.10 Faculty Member Dashboard.....     | 42 |
| 5.10.1 complaint details .....         | 42 |
| 5.11 Supervisor Dashboard.....         | 43 |
| 5.11.1 Analytic section.....           | 44 |
| 5.11.2 Activity Feed .....             | 44 |
| 5.11.3 Departmental Insight.....       | 45 |
| 5.12 Notifications Interface .....     | 46 |
| 5.13 Botpress Chatbot.....             | 46 |
| 5.14 Common Status Meanings .....      | 47 |

## **CHAPTER 6**

### **CONCLUSION**

|                            |    |
|----------------------------|----|
|                            | 48 |
| 6.1 Testing Strategy ..... | 49 |
| 6.2 Results .....          | 49 |
| 6.3 Conclusion .....       | 49 |
| 6.4 Future Work.....       | 50 |

# **CHAPTER 1**

## **INTRODUCTION**

Universities receive various student complaints related to academics, administration, facilities, transport, IT support, and other services. Traditional complaint handling methods such as verbal communication, paper forms, and emails often cause delays and poor tracking.

The Student Issue Tracker provides a centralized web-based solution for complaint management. Students can submit complaints online, upload files, track complaint status, and provide feedback after resolution. The system also offers AI-based category and priority suggestions, while staff members use role-based dashboards to manage complaints efficiently.

## **1.1 Background and Motivation**

Universities handle a large number of students concerns every day, ranging from academic issues and administrative delays to facility-related complaints and behavioral concerns. In many institutions, these complaints are managed through informal communication channels such as verbal discussions, emails, social media messages, or paper-based applications.

The Student Issue Tracker was developed to address these challenges by providing a centralized web-based platform that enables students to submit complaints in a structured manner. The system ensures transparency throughout the complaint lifecycle, allowing students to monitor progress while enabling university staff to manage issues more efficiently. The integration of role-based dashboards and AI-assisted suggestions further improves the overall complaint management process.

The motivation behind this project comes from the need to improve accountability, communication, and service quality within educational institutions. By digitizing complaint handling processes, the university can reduce delays, maintain accurate records, and ensure that every complaint receives appropriate attention and resolution.

The academic environment depends on timely communication between students and the offices responsible for support services. When a complaint is submitted through an unstructured channel, the student may not know who received it, whether it was forwarded to the correct authority, or how long the response will take. This uncertainty weakens trust in institutional services. A digital issue tracker reduces this uncertainty by converting each concern into a recorded case with

category, priority, attachment, status, assigned role, and activity history. The motivation of the project is therefore not only to computerize complaint submission, but also to create a transparent communication model where students and staff can follow the same verified record from submission to closure.

In the university context, complaints are not limited to one department or one type of service. A student may face an academic issue related to course registration, a facility issue related to classrooms or laboratories, an administrative issue related to documents, or a behavior-related issue that requires careful handling by responsible authorities. Because these categories involve different offices, a common platform becomes necessary. The Student Issue Tracker supports this need by giving each issue a defined digital identity. Once the complaint is recorded, it can be viewed, reviewed, assigned, and updated without depending on memory, paper movement, or informal personal communication.

The motivation is also connected with student confidence. When students believe that complaints disappear after submission, they may stop reporting problems or may use public and informal channels that make resolution more difficult. A transparent tracker encourages students to report issues through the official process because it gives them visible progress, status updates, and a final feedback option. For staff members, the same transparency reduces repeated inquiries and makes it easier to justify decisions. This mutual visibility is one of the major reasons for designing the project as a complete complaint lifecycle system rather than a simple complaint form.

## **1.2 Problem Statement**

Many universities still lack a centralized mechanism for managing student complaints. As a result, complaints are often lost, ignored, delayed, or handled inconsistently. Students frequently remain unaware of the status of their submitted issues, while administrative staff face difficulties in maintaining records and monitoring progress, while administrative staff face difficulties in maintaining records and monitoring progress in an efficient manner

The absence of a structured workflow creates confusion regarding responsibilities and decision-making. Departments may struggle to assign complaints to the appropriate personnel, resulting in inefficiencies and poor service delivery. Furthermore, traditional complaint management approaches do not provide sufficient reporting, analytics, or historical records for institutional improvement.

The Student Issue Tracker aims to solve these problems by introducing a digital complaint management system that supports complaint submission, tracking, assignment, resolution, notifications, reporting, and feedback collection through a unified platform.

The implemented system responds to these issues by defining a clear complaint lifecycle. A student submits an issue with supporting details, the platform suggests a suitable category and priority, and the complaint is routed to the responsible authority according to institutional rules. Academic and behavior-related matters are directed to the HOD, while administrative, facilities, and other concerns are directed to DSA. Accepted cases can then be assigned to faculty members or relevant staff, resolved with evidence, and closed only after review. This workflow addresses the core problem of responsibility, because each action is connected with a role and a recorded status rather than informal verbal follow-up.

Manual complaint handling also creates reporting problems. If records are stored in different email accounts, paper files, or personal notes, management cannot easily measure how many complaints were received, which departments were involved, how long resolution took, or which issue types occurred most frequently. Without this information, planning becomes reactive. The proposed system solves this weakness by maintaining structured complaint data that can be searched, filtered, summarized, and converted into visual reports. This makes the complaint process useful not only for individual resolution but also for institutional monitoring.

Another part of the problem is communication between roles. Students, HOD users, DSA users, faculty members, and supervisors all need different information at different stages. If the system exposes the same interface to every user, it becomes confusing and insecure. If it hides too much information, users cannot complete their responsibilities. The Student Issue Tracker addresses this by using role-based dashboards and access patterns. Each role receives the actions required for its part of the workflow, which improves usability while protecting complaint information from unnecessary exposure.

### **1.3 Project Objectives**

The primary objective of this project is to design and implement a comprehensive complaint management system for universities. The system seeks to streamline complaint submission, improve transparency, and enhance communication between students and university staff. Specific objectives include secure authentication, AI-assisted complaint categorization, role-based

dashboards, automated routing mechanisms, complaint tracking, activity logging, notification management, analytics generation, chatbot integration, and feedback collection. The project also focuses on maintaining data integrity through a normalized database structure and scalable architecture.

Another objective is to make the complaint process easier for both technical and non-technical users. Students should be able to submit issues without visiting multiple offices, while staff members should be able to filter, search, review, and update cases from a single dashboard. The system also aims to support evidence-based administration through complaint histories, activity logs, weekly reports, and visual analytics. These features help management identify repeated service problems, evaluate departmental response patterns, and plan corrective action. In this way, the project combines operational workflow support with decision-making support.

The objectives further include improving consistency in decision-making. Category-based routing ensures that complaints are not sent randomly, while priority suggestions help staff identify issues that may require faster attention. Activity logs provide a record of status changes and assignment actions, supporting accountability in case a complaint is delayed or reopened. The feedback mechanism allows students to evaluate the resolution experience after closure. Together, these objectives create a system that supports submission, review, action, closure, and evaluation as connected parts of one process.

## **1.4 Scope of the Project**

The scope of the Student Issue Tracker covers all major activities associated with complaint management. Students can register, log in, submit complaints, upload attachments, track complaint progress, and provide feedback after resolution. University staff including HODs, DSA representatives, faculty members, and supervisors can review, assign, monitor, and resolve complaints through dedicated dashboards across multiple university departments.

The project includes reporting features, complaint analytics, notification systems, AI-assisted recommendations, chatbot support, and activity logs. However, the current implementation is limited to a web-based environment and does not include mobile applications, advanced machine learning models, or multi-campus deployments. These enhancements are considered future development opportunities.

Within the current scope, the system focuses on a university-level deployment where complaint categories, staff roles, and review responsibilities are clearly defined. It covers the complete path from account authentication to complaint closure and student feedback. The scope also includes file attachments through Supabase Storage, profile management, searchable complaint queues, modal-based complaint details, notification records, and guided assistance through chatbot and voice-assistant widgets. The project does not attempt to replace all university management systems; rather, it concentrates on the specific problem of structured issue reporting and resolution.

The scope also includes the practical technologies required to make the platform usable in a real academic environment. Supabase Authentication is used for secure user access, Supabase PostgreSQL stores complaint and role information, and Supabase Storage supports uploaded evidence. Next.js API Routes provide the server-side interface for operations such as complaint submission, profile updates, staff actions, and reporting data. Botpress and browser speech synthesis are included as support tools to guide users and improve accessibility. These elements keep the project focused on a working web-based implementation rather than a purely theoretical model.

## **1.5 Significance of the Project**

The significance of the Student Issue Tracker lies in its ability to improve operational efficiency and student satisfaction. For students, the system provides a transparent and reliable mechanism for reporting concerns. For university management, it offers improved visibility into complaint trends and departmental performance for better decision-making.

The platform promotes accountability by recording every action performed within the complaint lifecycle. It also enables data-driven decision-making through analytics and reporting capabilities. As a result, the institution can identify recurring issues, allocate resources more effectively, and continuously improve service quality.

The project is significant because complaint records can become a source of institutional learning. When issues are stored, categorized, and analyzed over time, management can recognize patterns that are difficult to see through manual communication. For example, frequent facility complaints may indicate maintenance gaps, repeated academic complaints may reveal scheduling or communication problems, and delayed resolutions may show where additional staff coordination



is required. The system therefore supports both immediate service recovery for students and long-term quality improvement for the university.

For university staff, the significance is equally practical. A structured dashboard reduces the need to manually search through emails or written applications. Staff can view pending, accepted, assigned, resolved, and closed complaints according to their role. Supervisory users can observe workload and resolution progress through summaries and reports. This improves administrative control because unresolved complaints are visible and cannot easily be forgotten. The system therefore contributes to accountability, service transparency, and measurable improvement in student support services.

## **1.6 Methodology Overview**

The development of the Student Issue Tracker follows the System Development Life Cycle (SDLC) methodology. Requirements were gathered through an analysis of existing complaint management processes and stakeholder needs. System analysis was conducted to identify functional and non-functional requirements throughout the project lifecycle.

During the design phase, UML diagrams, data flow diagrams, entity relationship diagrams, and database schemas were prepared. The implementation phase utilized modern web technologies including Next.js, React, Supabase, and supporting libraries. Testing was conducted at multiple levels to ensure reliability, usability, and performance. Finally, documentation was prepared to support deployment, maintenance, and future enhancements.

The methodology also emphasized incremental validation of the implemented features. Authentication, complaint submission, routing, dashboards, staff assignment, evidence upload, feedback, analytics, chatbot support, and reporting were treated as separate functional areas during development and testing. This made it possible to verify each module before combining it into the final workflow. The use of Next.js for the interface and API routes, together with Supabase for authentication, database management, and storage, provided a practical full-stack structure for implementing and evaluating the system.

The SDLC approach was suitable because the project required careful movement from problem identification to implemented software. During requirement analysis, the major actors and complaint categories were identified. During system design, workflows, diagrams, database relationships, and user interfaces were planned. During implementation, each module was

developed according to the selected technology stack. During testing, functional behavior was checked for major tasks such as login, complaint creation, status update, assignment, evidence upload, feedback, search, and dashboard display. This staged process reduced development risk and helped ensure that the final system matched the intended workflow.

## **1.7 Limitations**

Despite providing significant improvements over traditional complaint management methods, the current version of the Student Issue Tracker has several limitations. The system operates exclusively as a web application and does not provide dedicated mobile applications for Android or iOS devices.

AI functionality is based primarily on rule-based and token-overlap approaches rather than advanced machine learning models. Real-time notification mechanisms are not fully implemented, and email delivery depends on external SMTP services. Furthermore, certain advanced workflow automation and escalation features remain outside the scope of the current implementation.

These limitations define the boundary of the current final year project. The rule-based AI assistance can support category and priority suggestions, but it cannot understand every complex complaint with the accuracy of a trained institutional model. Similarly, the web-based interface is responsive, but a dedicated mobile application could provide stronger offline access, push notifications, and device-level usability in future work. The present version is therefore best understood as a functional and extensible complaint management platform that establishes the core workflow and leaves advanced automation for later enhancement.

The limitations do not reduce the usefulness of the implemented system, but they clarify what should be improved in future versions. Real-time notifications, advanced AI classification, multi-campus support, stronger escalation policies, and dedicated mobile applications can extend the platform further. Integration with official university portals could also reduce duplicate account management and make authentication more centralized. These future directions are possible because the current architecture separates presentation, application logic, and data management, making the system easier to extend after the initial deployment.

## **1.8 System Architecture Overview**

The Student Issue Tracker follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer. The Presentation Layer provides user interfaces, dashboards,

forms, chatbot integration, and accessibility features. The Application Layer contains business logic, workflow management, complaint routing, AI recommendation services, and notification mechanisms.

The Data Layer utilizes Supabase services including PostgreSQL databases, authentication modules, storage services, and security policies. This architecture promotes scalability, maintainability, and separation of concerns while ensuring secure and efficient handling of complaint-related information across the platform.

The architecture supports future enhancements by allowing additional modules and integrations to be incorporated without significant redesign. This makes the system suitable for long-term institutional use and expansion.

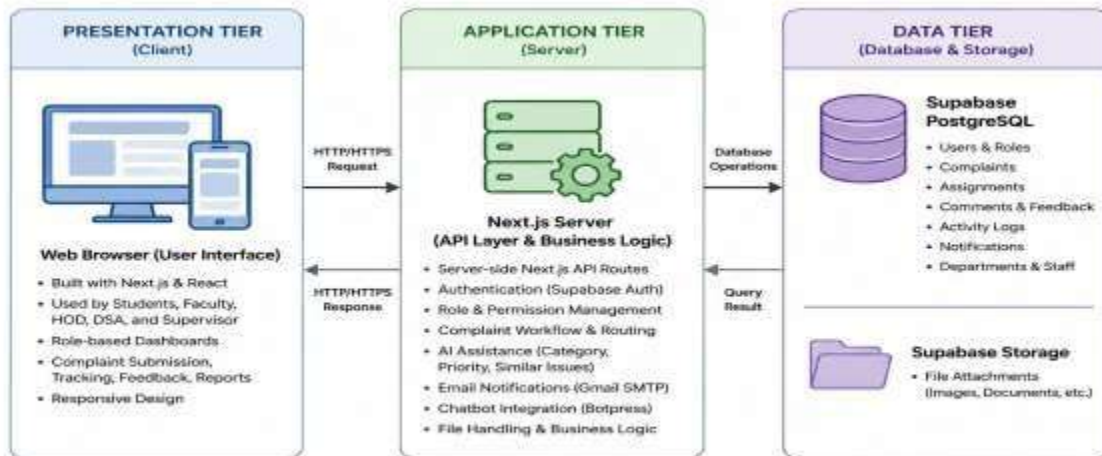
The three-tier structure is suitable for the project because it separates user interaction, business processing, and stored data. Students and staff interact with the presentation layer through role-specific pages and dashboards. The application layer handles routing decisions, validation, dashboard summaries, AI utilities, notification records, and API communication. The data layer maintains users, complaints, assignments, attachments, feedback, activity logs, and related institutional information. This separation improves maintainability because changes in one layer can be introduced with limited impact on the others, which is important for future features such as stronger escalation rules, improved AI models, and mobile clients.

From a user-experience perspective, the presentation layer is responsible for keeping the workflow understandable. Students need a direct path for submission and tracking, while staff users need queues, summaries, filters, details, and action buttons. The application layer coordinates these actions so that status transitions remain controlled and role-specific. The data layer preserves the final record of what happened. This relationship between layers makes the system dependable because the user interface does not directly manipulate institutional records without passing through application logic and validation.

Security is also supported by the architecture. Authentication confirms user identity, role checks restrict access to the correct dashboard actions, and database policies protect sensitive complaint information. Attachments are stored separately from textual complaint records, allowing evidence files to be managed through storage services while complaint metadata remains in relational tables. This structure is important for a complaint management system because cases may contain

personal, academic, or administrative details that should only be visible to authorized users. The architecture therefore supports both functionality and responsible information handling.

Overall, Chapter 1 establishes the need for the Student Issue Tracker by connecting institutional complaint-handling problems with the implemented system's purpose, scope, and architecture. The project is motivated by real delays and visibility gaps in manual complaint processes. It responds through a web-based platform that supports role-based access, category routing, AI-assisted suggestions, dashboards, attachments, reporting, feedback, and guided assistance. These foundations prepare the reader for the later chapters, where feasibility, requirements, design diagrams, implementation details, user operations, testing, and future work are discussed in greater depth.



**Figure 1.1**  
Three-Tier Architecture of Student Issue Tracker

**Figure 1.1: Three-tier architecture of Student Issue Tracker**

# **CHAPTER 2**

## **SYSTEM ANALYSIS**

System analysis is a critical phase of the System Development Life Cycle (SDLC) that focuses on understanding the existing environment, identifying user requirements, examining operational challenges, and defining the functionality expected from the proposed solution. The primary purpose of system analysis is to ensure that the developed system effectively addresses real-world problems while meeting stakeholder expectations.

In many universities, complaint handling procedures are still managed through manual methods such as paper forms, verbal communication, emails, and informal communication channels. These methods often lead to delays, poor record management, lack of transparency, and difficulty in tracking complaint progress. Students frequently experience frustration because they cannot determine whether their complaints have been reviewed, assigned, or resolved.

The Student Issue Tracker was proposed to overcome these limitations by providing a centralized complaint management platform. The system enables students to submit complaints electronically, track progress, receive notifications, and communicate through a structured workflow. Administrative authorities can efficiently review, assign, monitor, and resolve complaints while maintaining complete records.

The analysis process focused on identifying stakeholder requirements, studying existing complaint handling practices, evaluating technical and operational constraints, and determining the overall scope of the system. Particular attention was given to role-based workflows, complaint routing rules, AI-assisted complaint guidance, chatbot support, voice notifications, reporting features, and complaint lifecycle management.

The outcome of the analysis phase provides a strong foundation for system design and implementation by clearly defining what the system must achieve and how it should support university complaint management operations.

## **2.1 Feasibility Study**

### **2.1.1 Technical Feasibility**

The proposed Student Issue Tracker is technically feasible because it is built using modern and widely accepted web technologies. The frontend utilizes Next.js and React to provide an interactive, responsive, and user-friendly interface. These technologies offer excellent

performance, maintainability, and scalability for modern web applications.

Supabase is used as the backend service provider. It offers PostgreSQL database support, authentication services, secure storage, and Row Level Security policies. These capabilities reduce development complexity while providing enterprise-level functionality.

Additional services such as Gmail SMTP, Botpress chatbot integration, and browser-based speech synthesis further enhance system functionality. The availability of documentation, active developer communities, and continuous platform support ensures that technical challenges can be addressed efficiently.

Therefore, the required software tools, frameworks, infrastructure, and development resources are readily available, making the project technically feasible.

### **2.1.2 Operational Feasibility**

Operational feasibility evaluates whether the proposed system can function effectively within the university environment. The Student Issue Tracker closely follows existing complaint handling procedures while introducing automation and transparency.

Students can submit complaints and monitor their progress without repeatedly contacting university staff. HODs and DSA representatives can review complaints according to predefined routing policies, while faculty members can manage assigned issues efficiently.

The interface design uses familiar elements such as forms, dashboards, cards, notifications, search bars, and status indicators. These familiar patterns reduce training requirements and improve user acceptance. Consequently, the system can be integrated into existing university operations with minimal disruption.

### **2.1.3 Economic Feasibility**

Economic feasibility determines whether the expected benefits justify the development and maintenance costs. The Student Issue Tracker is economically feasible because it primarily relies on open-source technologies and low-cost cloud services. The use of Next.js, React, JavaScript, and Supabase significantly reduces licensing expenses. Furthermore, free-tier development services can be used during the implementation phase.

The system also reduces administrative workload, minimizes paperwork, improves response efficiency, and enhances student satisfaction. These operational benefits contribute to a positive cost-benefit ratio and justify investment in the system.

### 2.1.4 Schedule Feasibility

Schedule feasibility examines whether the project can be completed within the available timeframe. The Student Issue Tracker was divided into manageable modules including authentication, complaint submission, complaint routing, notifications, analytics, reporting, chatbot integration, and profile management.

The modular structure allows independent development and testing of each component. This approach minimizes risk, simplifies project management, and supports phased implementation.

As a result, the project schedule remains practical and achievable within the academic development timeline.

### 2.1.5 Legal and Security Feasibility

The Student Issue Tracker processes sensitive complaint information; therefore, legal and security considerations are extremely important. The system implements secure authentication, role-based access control, protected API routes, activity logging, and Row Level Security policies.

Anonymous complaint submission is supported to encourage reporting of sensitive issues. User permissions restrict access to authorized records only, while environment variables protect confidential credentials.

These security measures help maintain privacy, accountability, and compliance with institutional policies governing student information.

**Table 2.1: Feasibility Summary**

| Feasibility Type           | Assessment                                                            | Rating |
|----------------------------|-----------------------------------------------------------------------|--------|
| Technical Feasibility      | Modern full-stack tools and managed database services are available.  | High   |
| Operational Feasibility    | Workflow matches real university complaint handling responsibilities. | High   |
| Economic Feasibility       | Open-source and free-tier services reduce cost.                       | High   |
| Schedule Feasibility       | System is modular and can be implemented in phases.                   | High   |
| Legal/Security Feasibility | Authentication, RBAC, RLS and logs protect data.                      | High   |



## **2.2 Data Gathering Techniques**

Data gathering activities were conducted to understand existing complaint handling practices and identify the requirements of students and university staff. Multiple techniques were used to ensure accurate and reliable information collection.

The collected information helped identify operational challenges, workflow requirements, desired system features, and opportunities for process improvement. The findings obtained through these techniques directly influenced the design and functionality of the proposed system.

### **2.2.1 Questionnaires**

Questionnaires were distributed among students to collect opinions regarding existing complaint management practices. The questions focused on complaint submission methods, response times, complaint tracking capabilities, duplicate complaint issues, and desired digital features.

The responses revealed that students require a transparent complaint tracking mechanism, faster communication, status updates, and the ability to submit complaints electronically. Many participants also highlighted the need for anonymous complaint submission options.

The questionnaire findings provided valuable insight into user expectations and contributed significantly to system requirement identification.

### **2.2.2 Sampling and Observations**

Observation was conducted to study how complaints are currently managed within educational institutions. Existing workflows were analyzed to identify delays, inefficiencies, and record management problems.

The observations indicated that manual complaint handling often leads to incomplete records, delayed responses, inconsistent decision-making, and limited visibility into complaint progress. Duplicate complaints were also found to increase staff workload unnecessarily.

These observations reinforced the need for a centralized complaint management platform capable of automating workflow activities and improving transparency.

### **2.2.3 Interviews**

Interviews were conducted with academic and administrative stakeholders to understand complaint review procedures, assignment responsibilities, and resolution requirements.

The discussions revealed that complaints should be reviewed before assignment, invalid complaints should be rejected, and category-based routing should be implemented. Stakeholders

emphasized the importance of accountability, feedback collection, workload monitoring, and reporting capabilities.

The interview findings played an important role in defining complaint workflows and role-based responsibilities.

## 2.3 Functional Requirements

Functional requirements describe the specific services and operations the system must provide. These requirements define how users interact with the system and how complaint information moves through the complaint lifecycle.

The Student Issue Tracker supports complaint registration, complaint creation, AI-assisted suggestions, category-based routing, complaint review, assignment, resolution, closure, feedback collection, notifications, reporting, chatbot assistance, and profile management.

Each user role performs specific functions according to organizational responsibilities. Students submit and track complaints, HODs and DSA officers review and assign complaints, faculty members resolve issues, and supervisors manage overall operations and reporting activities.

**Table 2.2: Functional Requirements**

| FR-ID | User Role      | Requirement                                                                                                      |
|-------|----------------|------------------------------------------------------------------------------------------------------------------|
| FR-01 | Student        | Register and login using Supabase Auth.                                                                          |
| FR-02 | Student        | Submit complaint with title, description, category, priority, anonymous option and attachments.                  |
| FR-03 | Student        | Receive AI-assisted category, priority and similar complaint suggestions.                                        |
| FR-04 | Student        | Search complaints, view complaint details, track status and submit rating/feedback after resolution.             |
| FR-05 | HOD            | View Academic and Behavior-related complaints routed to HOD for assigned department.                             |
| FR-06 | HOD            | Review/reject complaints and assign valid complaints to Faculty Members.                                         |
| FR-07 | DSA            | Review Administrative, Facilities and Other complaints, assign accepted cases, and finalize resolved complaints. |
| FR-08 | Faculty Member | View assigned complaints, open details, contact students when available, upload evidence and mark resolved.      |

|       |            |                                                                                                                 |
|-------|------------|-----------------------------------------------------------------------------------------------------------------|
| FR-09 | Supervisor | Manage departments, staff, complaints, analytics and activity logs.                                             |
| FR-10 | System     | Generate notifications, email alerts, weekly reports, analytics, chatbot guidance and voice-assistant messages. |

## 2.4 Non-Functional Requirements

Non-functional requirements define the quality characteristics of the system. These requirements focus on performance, reliability, security, maintainability, scalability, usability, and privacy.

Security mechanisms protect complaint data through authentication and authorization controls.

Reliability ensures that complaint workflows continue even when supporting services experience temporary failures. Maintainability is achieved through modular design and organized project structure.

Scalability enables future growth in user numbers and complaint volume, while privacy features protect sensitive information through anonymous complaint options and restricted access control

**Table 2.3: Non-Functional Requirements**

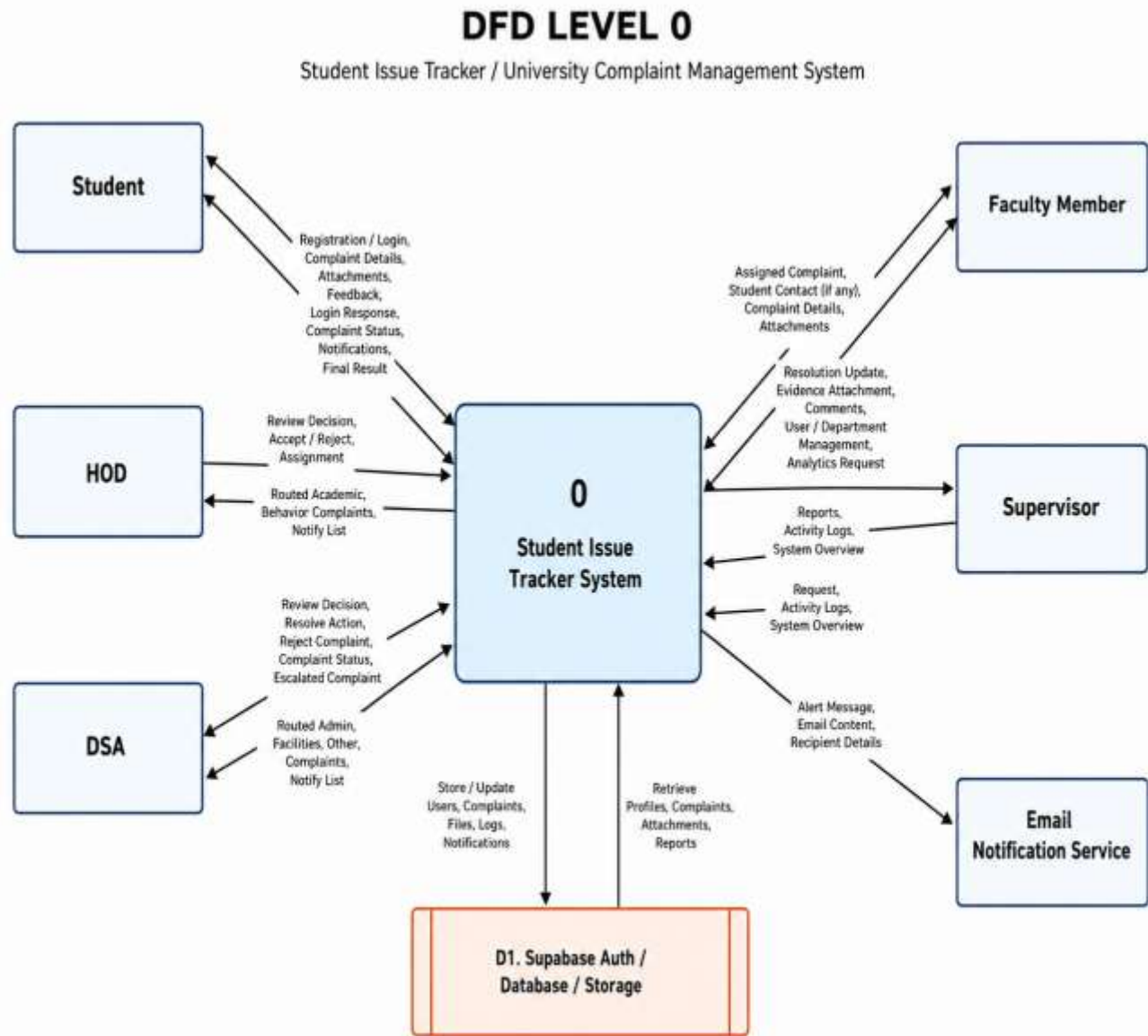
| NFR-ID | Category        | Requirement                                                                      |
|--------|-----------------|----------------------------------------------------------------------------------|
| NFR-01 | Security        | Supabase Auth, protected API routes, role checks and RLS policies.               |
| NFR-02 | Usability       | Simple dashboards, complaint cards, status badges and form validation.           |
| NFR-03 | Reliability     | Workflow actions continue safely even if email sending fails.                    |
| NFR-04 | Maintainability | Separate components, API routes, utilities and database schema.                  |
| NFR-05 | Scalability     | Supabase PostgreSQL supports structured growth of users and complaints.          |
| NFR-06 | Traceability    | Activity logs capture important complaint events.                                |
| NFR-07 | Privacy         | Anonymous submission option and scoped data access protect sensitive complaints. |

## 2.5 Data Flow Diagram - Level 0 (Context Diagram)

The Level 0 DFD shows the Student Issue Tracker as one main system that interacts with external users and services. Students provide registration details, login information, complaints, attachments, search or status requests, ratings, feedback, and chatbot queries.

On the other side, HOD, DSA, Faculty Members, and Supervisors send actions like reviewing complaints, assigning tasks, making decisions, updating resolution status, finalizing issues, managing records, and requesting reports.

The system also connects with external services. It uses Botpress for chatbot support, Gmail SMTP for sending email notifications, browser speech synthesis for voice alerts, and Supabase for authentication, database storage, and file management.



**Figure 2.1: Data Flow Diagram - Level 0 (Context Diagram)**

## 2.6 Data Flow Diagram - Level 1

The Level 1 DFD breaks the system into main internal processes such as authentication, complaint submission, AI suggestions, routing of complaints, review process, assignment of tasks, resolution handling, final closure of complaints, rating and feedback, notifications, analytics, search and filtering, profile management, chatbot support, voice assistant, and department/staff management.

This breakdown explains how data flows step by step — starting from a student submitting a complaint and moving through different processing stages until it is resolved, finalized, and stored for feedback and reporting.

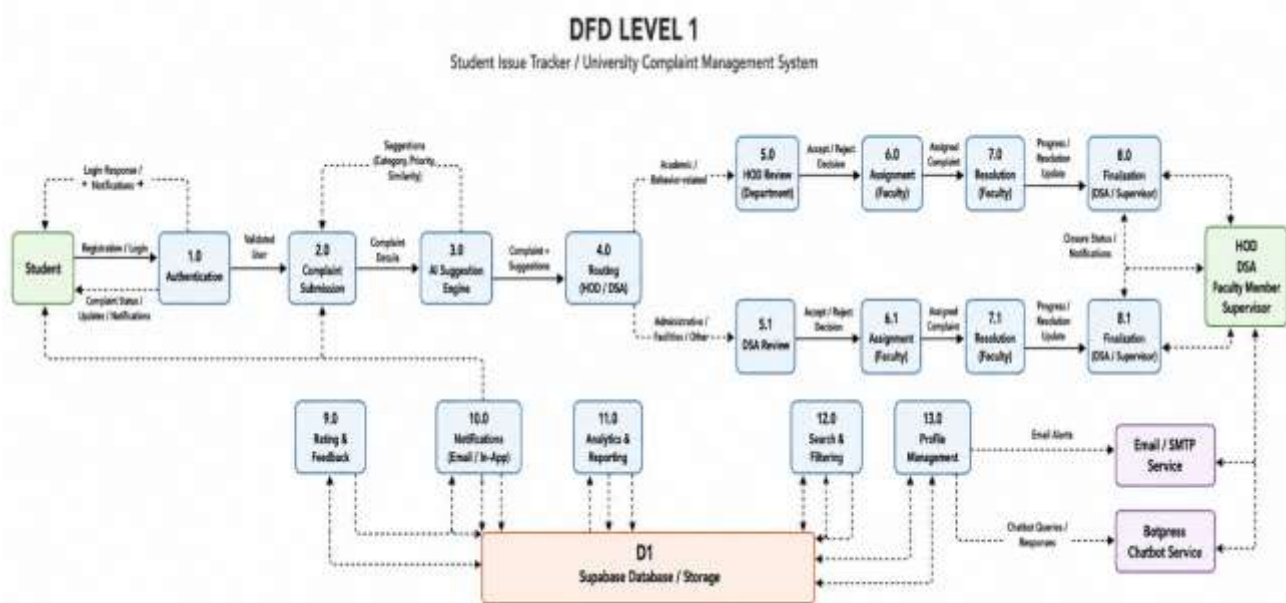


Figure 2.2: Data Flow Diagram - Level 1

**Table 2.4: DFD Level 1 Process Specification**

| Process                  | Input                                | Output                                                                                  | Data Store                        |
|--------------------------|--------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------|
| 1.0 Authentication       | Login/register data                  | Validated session and profile                                                           | Supabase Auth, profiles           |
| 2.0 Complaint Submission | Complaint details and files          | Complaint ID and submitted status                                                       | complaints, attachments, storage  |
| 3.0 AI Suggestion        | Complaint description                | Suggested category, priority, similarity score                                          | complaints                        |
| 4.0 Routing              | Category and complaint department    | Routed role: HOD for Academic/Behavior-related, DSA for Administrative/Facilities/Other | workflow defaults, complaints     |
| 5.0 Review               | Accept/reject action                 | Accepted or rejected status                                                             | complaints, activity_logs         |
| 6.0 Assignment           | Faculty member and deadline          | In Progress complaint                                                                   | complaints, profiles              |
| 7.0 Resolution           | Faculty evidence and resolved action | Resolved complaint with evidence/attachments                                            | complaint_attachments, complaints |
| 8.0 Finalization         | Close action                         | Closed complaint                                                                        | complaints                        |
| 9.0 Rating               | Rating and feedback                  | Saved feedback                                                                          | complaints                        |
| 10.0 Notifications       | Workflow event                       | Dashboard notification and email                                                        | notifications                     |
| 11.0 Analytics           | Complaint records                    | Charts and reports                                                                      | complaints                        |
| 12.0 Management          | Department/user details              | Updated records                                                                         | departments, profiles             |

# **CHAPTER 3**

## **SYSTEM DESIGN**



System design explains how the Student Issue Tracker components communicate and how the complaint workflow is represented through UML, database design, and normalization. The design connects the Next.js React frontend, role-aware dashboards, modal detail views, Next.js API routes, Supabase Auth, Supabase PostgreSQL, Supabase Storage, AI utilities, notification helpers, Gmail SMTP email service, Botpress integration, browser speech synthesis, and responsive UI behavior.

3.1 Proposed System and Its Features

The proposed Student Issue Tracker is designed as a centralized web-based complaint management platform that improves transparency, accountability, and efficiency in handling student issues. The system integrates modern web technologies, role-based access control, AI-assisted complaint guidance, automated routing, reporting, and notification services into a single platform.

The proposed solution addresses the limitations identified during system analysis by replacing manual complaint handling procedures with a structured digital workflow. Students can submit complaints, upload supporting documents, track complaint status, and provide feedback after resolution. Administrative users can review complaints, assign tasks, monitor progress, and generate reports.

The system follows a three-layer architecture consisting of Presentation Layer, Application Layer, and Data Layer. This architecture promotes scalability, maintainability, security, and future expansion opportunities.

Table 3.1: Logical Layers of Proposed System

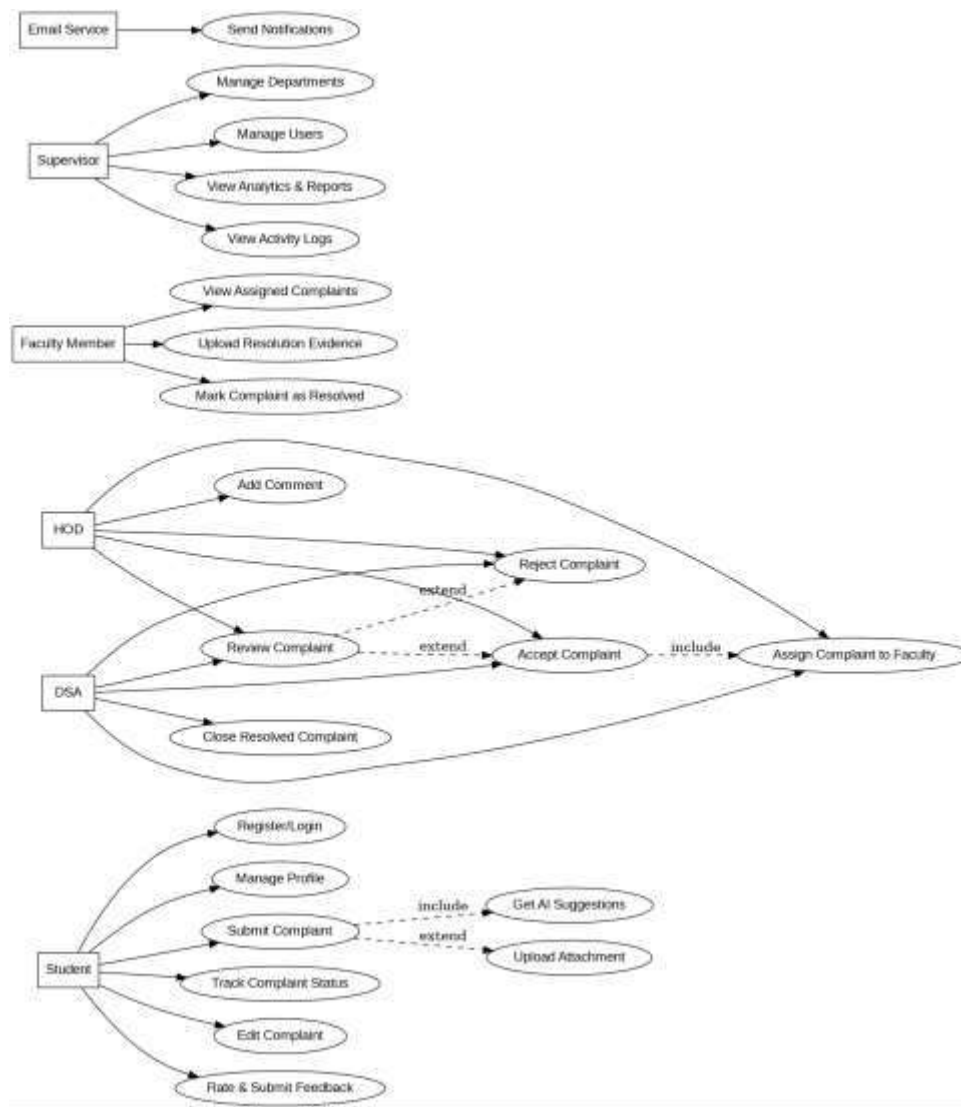
| Layer        | Technology                       | Responsibilities                                                                                                                         |
|--------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Presentation | Next.js + React + CSS animations | Welcome page, login, registration, dashboards, complaint form, cards, modal details, search, analytics, chatbot widget, voice assistant. |
| Application  | Next.js API Routes               | Authentication checks, RBAC, complaint workflow, AI utilities, emails, reports, analytics, comments, attachments, profile actions.       |
| Data         | Supabase PostgreSQL + Storage    | Profiles, departments, complaints, attachments, comments, notifications, routes, activity logs and files.                                |

### 3.2 Use Case Diagram

The Use Case Diagram provides a graphical representation of how different actors interact with the Student Issue Tracker. The primary actors include Student, HOD, DSA, Faculty Member, and Supervisor. External services such as Supabase, Gmail SMTP, Botpress, and Voice Assistant also support system operations.

Students can register, log in, submit complaints, upload attachments, receive AI suggestions, track complaint progress, and submit feedback. HODs and DSA can review complaints, reject invalid complaints, and approve valid complaints for assignment. and monitor complaint activities. Faculty Members resolve assigned complaints and upload evidence. Supervisors manage departments, staff accounts, analytics, and activity logs.

The Use Case Diagram helps stakeholders understand system boundaries, user responsibilities, and available system functions.



**Figure 3.1: UML Use Case Diagram for Student Issue Tracker**

### 3.3 Sequence Diagrams

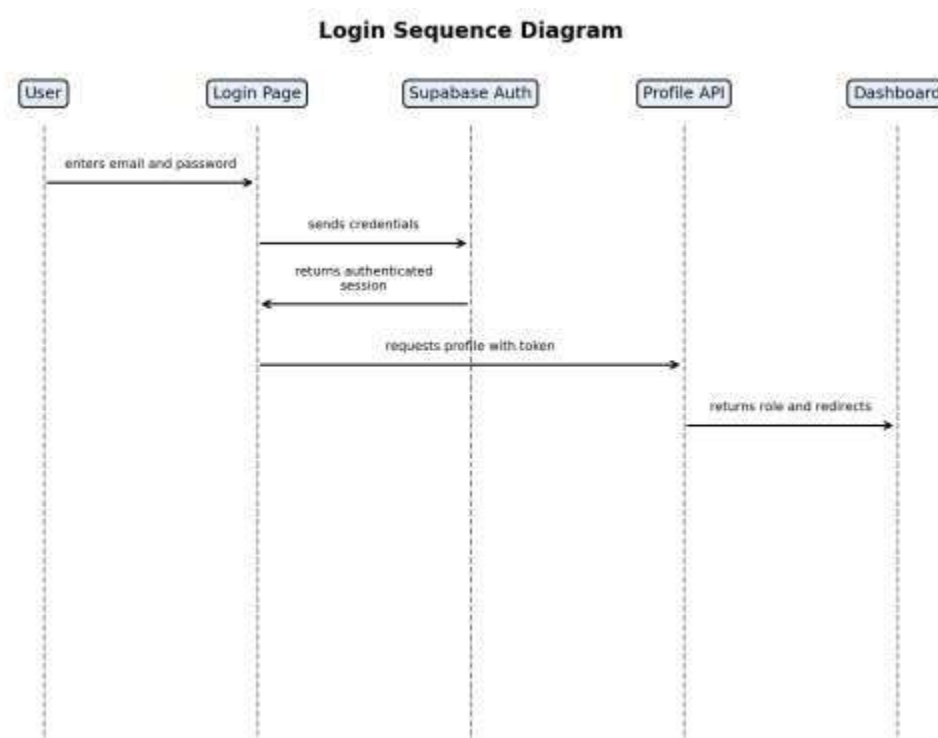
Sequence diagrams illustrate the chronological flow of interactions between users, interface components, APIs, and database services. These diagrams provide a detailed understanding of how requests are processed and how system components communicate with each other. The Student Issue Tracker contains multiple sequence diagrams representing critical workflows. These workflows include login authentication, complaint submission, AI-assisted suggestions, complaint review, assignment, resolution, and final closure.

Each sequence diagram demonstrates message flow, processing order, and interaction between

frontend and backend components. These diagrams are useful for validating workflow correctness and implementation planning.

### 3.3.1 Sequence Diagram – User Login

This sequence begins when a user enters login credentials. Supabase Authentication validates the credentials and generates a session token. The system then retrieves profile information and identifies the user role. Based on the authenticated role, the user is redirected to the appropriate dashboard. The login process ensures secure access while enforcing role-based permissions throughout the application.

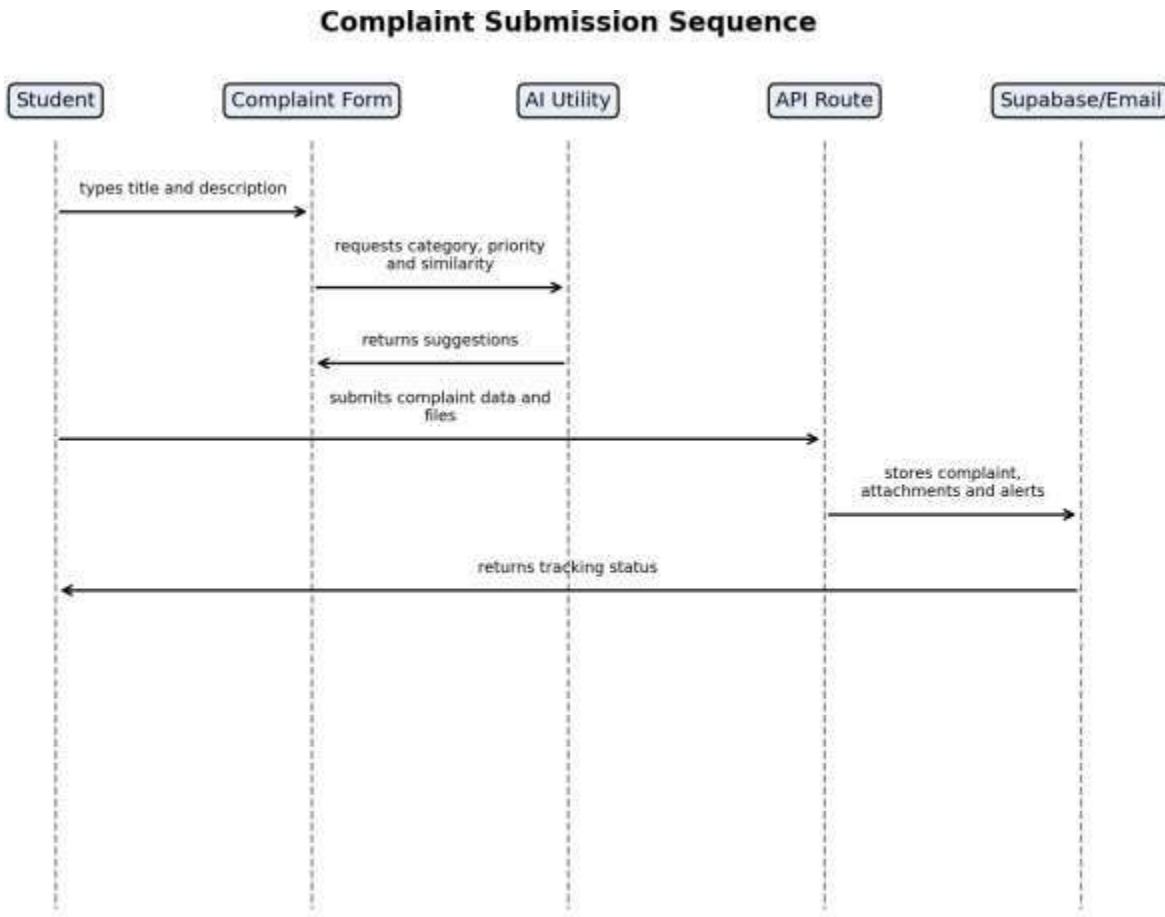


**Figure 3.2: Sequence Diagram - User Login**

### 3.3.2 Sequence Diagram – Complaint Submission with AI Assistance

The complaint submission workflow starts when a student enters complaint details. The AI assistance module analyzes the complaint description and suggests category and priority values. Similar complaints may also be identified.

After submission, complaint information is validated and stored in the database. The routing module determines whether the complaint should be sent to HOD or DSA according to category rules. Notifications and activity logs are automatically generated.

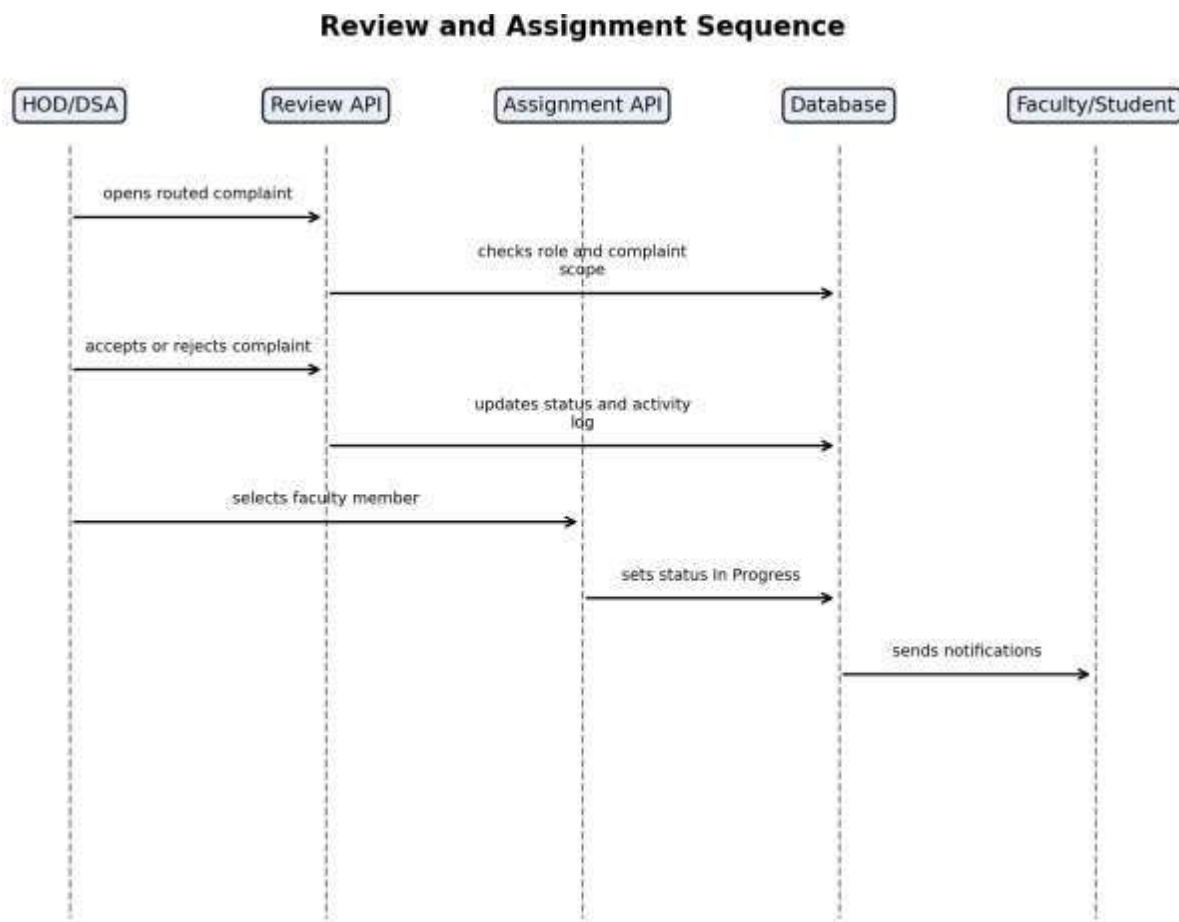


**Figure 3.3: Sequence Diagram - Complaint Submission with AI Assistance**

### 3.3.3 Sequence Diagram – Review and Faculty Assignment

This sequence describes how HOD or DSA reviews incoming complaints. Complaint details are examined to determine validity and scope. Invalid complaints may be rejected, while valid complaints are assigned to an appropriate faculty member.

Assignment actions update complaint status and generate notifications for both faculty members and students. Activity logs ensure complete traceability of workflow actions.

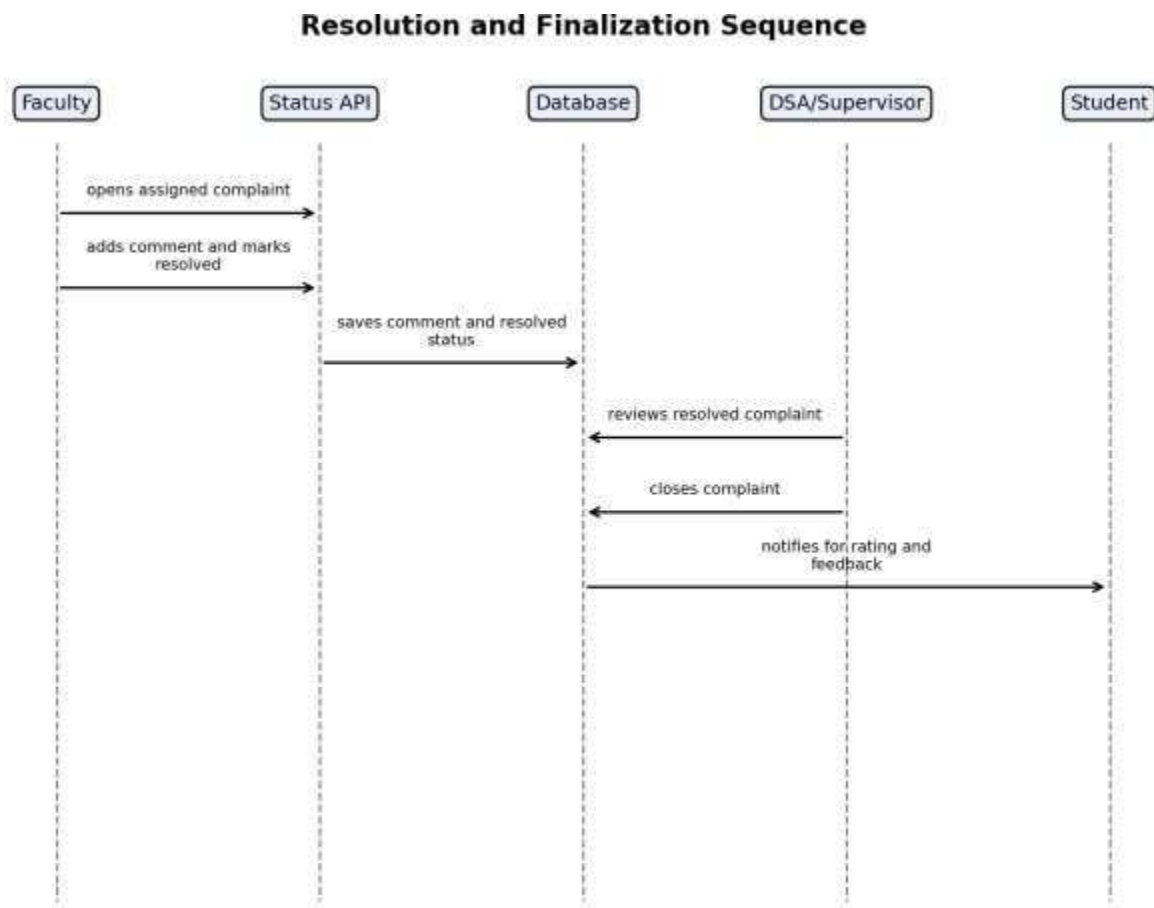


**Figure 3.4: Sequence Diagram - Review and Faculty Assignment**

### 3.3.4 Sequence Diagram – Resolution and Finalization

Faculty members review assigned complaints, communicate with students when necessary, upload supporting evidence, and mark complaints as resolved. The DSA or Supervisor then verifies completion and finalizes the complaint.

After closure, students receive notifications and may submit ratings and feedback. This sequence ensures accountability throughout the complaint lifecycle.



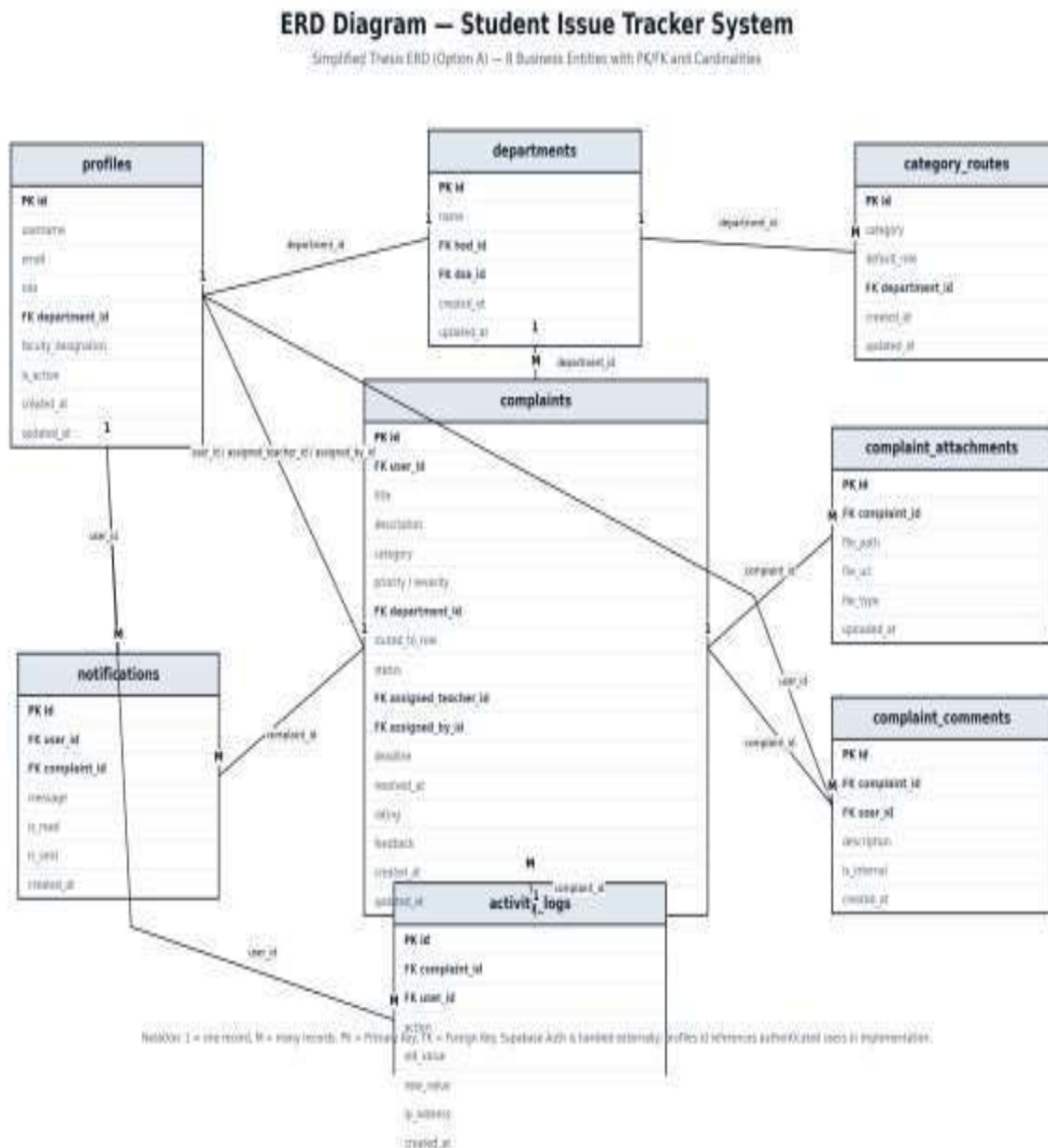
**Figure 3.5: Sequence Diagram - Resolution and Finalization**

### 3.4 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram illustrates the logical structure of the Student Issue Tracker database. It shows how profiles, departments, complaints, notifications, attachments, comments, category routes, and activity logs are interconnected.

The ERD helps developers understand database relationships, primary keys, foreign keys, and data dependencies. Proper database design supports efficient querying, reporting, workflow management, and security enforcement.

The design also minimizes redundancy by separating related information into dedicated tables. This improves maintainability and supports future expansion.



**Figure 3.6: Entity Relationship Diagram for Student Issue Tracker**

### 3.5 Relational Schema

The relational schema provides a tabular representation of database entities and their attributes. Each table contains a primary key and relevant fields required to support complaint management functionality.



Major tables include profiles, departments, complaints, complaint attachments, complaint comments, notifications, category routes, and activity logs. Relationships between tables are established through foreign keys to ensure referential integrity.

The schema serves as a blueprint for database implementation and supports efficient storage and Retrieval of information.

**Table 3.2: Database Tables - Relational Schema**

| Table Name            | Attributes                                                                                                                                                                                                                                                                                 |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| profiles              | id PK, username, email, role, department_id FK, faculty_designation, is_active, created_at, updated_at                                                                                                                                                                                     |
| departments           | id PK, name, hod_id FK, dsa_id FK, created_at, updated_at                                                                                                                                                                                                                                  |
| complaints            | id PK, user_id FK, title, description, category, severity, priority, suggested_category, suggested_priority, department_id FK, routed_to_role, is_anonymous, status, assigned_teacher_id FK, assigned_by_id FK, deadline, resolved_at, rating, feedback, escalated, created_at, updated_at |
| Complaint attachemnts | id PK, complaint_id FK, file_path, file_url, file_type, uploaded_at                                                                                                                                                                                                                        |
| complaint_comments    | id PK, complaint_id FK, user_id FK, description, is_internal, created_at                                                                                                                                                                                                                   |
| notifications         | id PK, user_id FK, complaint_id FK, message, is_read, is_sent, created_at                                                                                                                                                                                                                  |
| category_routes       | id PK, category, default_role, department_id FK, created_at, updated_at                                                                                                                                                                                                                    |
| activity_logs         | id PK, complaint_id FK, user_id FK, action, old_value, new_value, ip_address, created_at                                                                                                                                                                                                   |

### 3.6 Normalization to 3NF

Database normalization was performed to reduce redundancy and improve data consistency. TheStudent Issue Tracker database satisfies First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

In First Normal Form, all attributes contain atomic values and repeating groups are eliminated. Separate tables are used for attachments, notifications, comments, and activity logs.

In Second Normal Form, every non-key attribute depends entirely on the primary key of its table. Partial dependencies are removed to improve consistency.

In Third Normal Form, transitive dependencies are eliminated by separating departments, profiles, routes, notifications, and logs into dedicated tables. This design reduces duplication and simplifies maintenance.

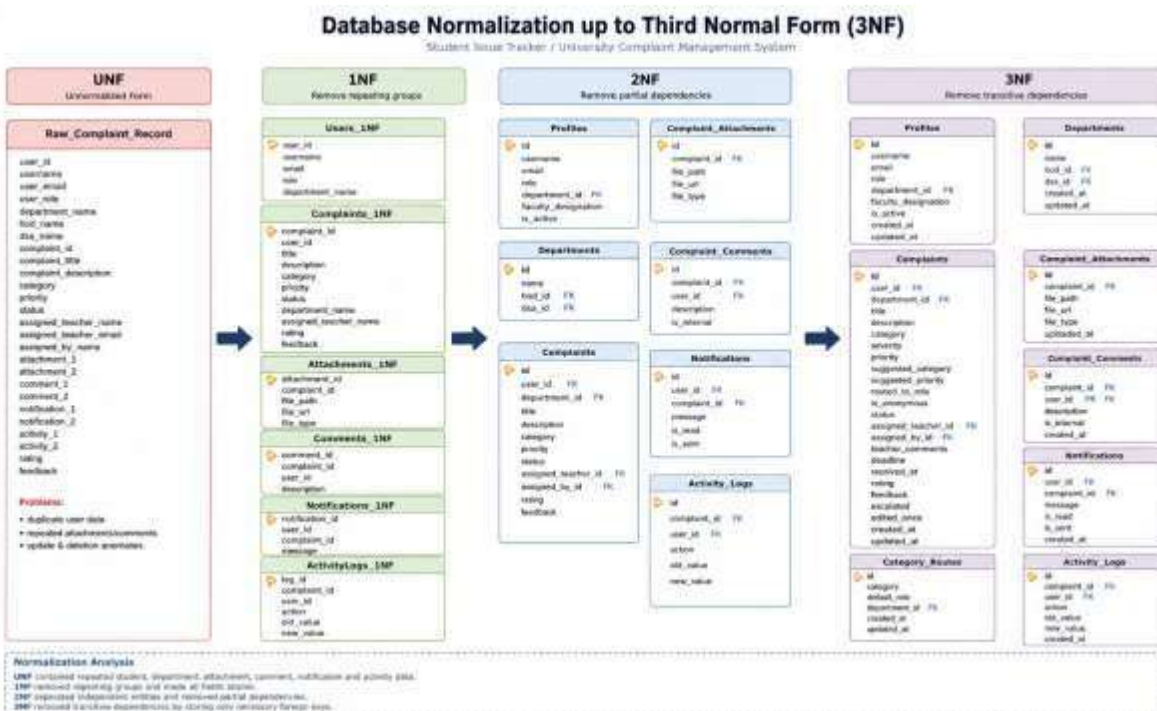


Figure 3.7: Normalization to Third Normal Form

**Table 3.3: Normalization Summary**

| Normal Form | Application in Project                                                                         |
|-------------|------------------------------------------------------------------------------------------------|
| 1NF         | Atomic values; attachments, comments and notifications separated from complaints.              |
| 2NF         | Each non-key attribute depends on its table primary key.                                       |
| 3NF         | No transitive dependency; profile, department, route, notification and log data are separated. |

# **CHAPTER 4**

## **SYSTEM DEVELOPMENT AND IMPLEMENTATION**

This chapter explains how the Student Issue Tracker was implemented using the selected technology stack. It covers authentication, role-based access control, complaint workflow, AI assistance, API endpoints, notifications, email, attachments, analytics, Botpress integration, draggable voice assistant, responsive UI behavior, search, profile editing, and security measures.

## **4.1 Technology Stack**

The Student Issue Tracker was developed using a modern full-stack technology stack that provides scalability, maintainability, security, and efficient performance. The selected technologies were chosen after evaluating project requirements, ease of integration, community support, and compatibility with cloud-based deployment environments.

The frontend of the application was implemented using Next.js App Router and React. Next.js provides server-side rendering, dynamic routing, API route support, and optimized page loading. React enables the development of reusable user interface components, reducing code duplication and improving maintainability. Custom CSS animations, responsive layouts, and Next/Image optimization were used to enhance user experience and application performance.

The backend logic was implemented using Next.js API Routes. This approach allows frontend and backend functionality to remain within a single project structure while maintaining separation of concerns. API routes handle authentication, complaint processing, assignment workflows, reporting services, and notification management.

Supabase PostgreSQL was selected as the primary database management system because it provides relational database capabilities, secure authentication services, cloud storage, and Row Level Security (RLS) features. Supabase significantly reduced backend infrastructure complexity while maintaining enterprise-level security.

Additional technologies such as Gmail SMTP, Botpress Cloud Webchat, Lucide React icons, Browser Speech Synthesis APIs, bcryptjs password utilities, and Supabase Storage further enhanced system functionality and user experience.

**Table 4.1: Technology Stack Summary**

| Area                         | Technology                                                                          |
|------------------------------|-------------------------------------------------------------------------------------|
| Frontend                     | Next.js App Router, React, CSS animations, Next/Image, responsive custom components |
| Backend                      | Next.js API Routes                                                                  |
| Database                     | Supabase PostgreSQL                                                                 |
| Authentication               | Supabase Auth                                                                       |
| Storage                      | Supabase Storage complaint-attachments bucket                                       |
| Analytics Visualization      | Custom SVG status trend chart and dashboard stat cards                              |
| Icons/UI                     | lucide-react icons and custom CSS components                                        |
| Email                        | Gmail SMTP through Node TLS helper                                                  |
| Chatbot                      | Botpress Cloud Webchat with draggable floating widget                               |
| Assistant / Security Helpers | Browser speech synthesis, bcryptjs and password validation                          |

## 4.2 Project Structure

| Path                       | Purpose                                                                                                            |
|----------------------------|--------------------------------------------------------------------------------------------------------------------|
| src/app/page.jsx           | Server-rendered welcome page with Issue Tracker branding, circular logo and CSS animation.                         |
| src/app/login/page.jsx     | Login screen using Supabase Auth.                                                                                  |
| src/app/register/page.jsx  | Registration screen.                                                                                               |
| src/app/dashboard/page.jsx | Unified role-aware dashboard with lazy-loaded components, search, profile menu, analytics and floating assistants. |
| src/app/api/complaints     | Complaint list, creation, analytics, reports, notifications and workflow APIs.                                     |
| src/components             | Reusable UI components such as ComplaintForm, ComplaintCard, AIPanel, badges, Botpress chatbot and VoiceRobot.     |
| src/lib                    | Auth helpers, API helpers, workflow routing rules, AI utility functions, email helpers and Supabase client.        |
| supabase/schema.sql        | Database schema, enums, indexes, RLS policies and storage bucket setup.                                            |

**Table 4.2: Important Project Files and Modules**

### **4.3 Authentication and RBAC Implementation**

Authentication is one of the most critical components of the Student Issue Tracker because all complaint-related operations require secure user identification. The system implements authentication using Supabase Auth, which provides secure user registration, login, session management, and access token generation.

When a user logs into the application, Supabase validates the provided credentials and creates an authenticated session. The generated access token is attached to protected API requests and verified by server-side routes before allowing access to application resources. This mechanism prevents unauthorized users from accessing complaint information.

Role-Based Access Control (RBAC) was implemented to ensure that users only access functionality relevant to their responsibilities. Each user profile contains a role attribute that determines available permissions throughout the application.

Students can create complaints, track complaint status, upload attachments, and provide ratings or feedback after resolution. Faculty Members can only view complaints assigned to them and are responsible for updating resolution status and uploading evidence.

HOD users can review Academic and Behavior-related complaints routed to their department. DSA users manage Administrative, Facilities, and Other complaint categories. Supervisors possess broader permissions allowing them to monitor complaints across departments, manage users, review reports, and oversee system activities.

The RBAC implementation improves security, prevents unauthorized actions, and ensures accountability by restricting access according to organizational responsibilities.

### **4.4 Complaint Workflow Implementation**

Authentication is one of the most critical components of the Student Issue Tracker because all complaint-related operations require secure user identification. The system implements authentication using Supabase Auth, which provides secure user registration, login, session management, and access token generation.

When a user logs into the application, Supabase validates the provided credentials and creates an authenticated session. The generated access token is attached to protected API requests and verified by server-side routes before allowing access to application resources. This mechanism prevents unauthorized users from accessing complaint information.

Role-Based Access Control (RBAC) was implemented to ensure that users only access functionality relevant to their responsibilities. Each user profile contains a role attribute that determines available permissions throughout the application.

Students can create complaints, track complaint status, upload attachments, and provide ratings or feedback after resolution. Faculty Members can only view complaints assigned to them and are responsible for updating resolution status and uploading evidence.

HOD users can review Academic and Behavior-related complaints routed to their department. DSA users manage Administrative, Facilities, and Other complaint categories. Supervisors possess broader permissions allowing them to monitor complaints across departments, manage users, review reports, and oversee system activities.

The RBAC implementation improves security, prevents unauthorized actions, and ensures accountability by restricting access according to organizational responsibilities.

## **4.5 AI Assistance Implementation**

Artificial Intelligence assistance was incorporated into the Student Issue Tracker to improve complaint quality and reduce errors during complaint submission. Instead of relying entirely on manual category selection, the system analyzes complaint descriptions and provides intelligent recommendations to students before final submission.

The AI module was implemented using lightweight JavaScript utilities that operate directly within the application. This approach eliminates the need for external machine learning servers while maintaining fast response times and low infrastructure costs. Because the AI logic runs within the existing application environment, deployment and maintenance remain simple and efficient.

Category prediction is performed using predefined keyword dictionaries associated with complaint categories such as Academic, Administrative, Facilities, Behavior-related, and Other. When a



student enters a complaint description, the system scans the text and compares detected keywords with category-specific keyword groups. Based on the matching score, the most appropriate complaint category is suggested.

Priority prediction is performed using an urgency analysis mechanism. The system evaluates words and phrases that indicate urgency, risk, deadlines, safety concerns, or critical situations. Keywords such as "urgent", "immediately", "harassment", "danger", "exam", "critical", "failure", and "broken" contribute to the overall priority score. Depending on the calculated score, the system recommends Low, Medium, or High priority levels.

To reduce duplicate complaint submissions, a similarity analysis component compares the newly entered complaint description with previously stored complaint records. Token-based comparison techniques are used to calculate overlap between complaint descriptions. The resulting similarity score helps identify related complaints and encourages students to review existing issues before creating duplicate submissions.

The AI assistance module improves complaint classification accuracy, reduces reviewer workload, and enhances the overall efficiency of the complaint management process.

## **4.6 Notification and Email Implementation**

An effective notification mechanism is essential for maintaining communication between students and administrative staff. The Student Issue Tracker includes both dashboard notifications and email notifications to ensure that users remain informed about complaint-related activities.

Whenever an important workflow event occurs, such as complaint submission, assignment, rejection, resolution, closure, or feedback submission, the system automatically generates notification records. These notifications are stored in the notifications table and displayed within the application dashboard.

Unlike traditional systems that rely heavily on sidebar notification badges, the Student Issue Tracker presents notifications within the Analytics section. This design reduces interface clutter while ensuring that users can easily access important updates.

Email notifications provide an additional communication channel for critical workflow events. The system uses Gmail SMTP services together with reusable helper functions to generate and deliver email messages. Email templates are dynamically populated with complaint information, status updates, and user-specific details.

To improve reliability, the email subsystem was designed using a fail-safe approach. If email credentials are unavailable or message delivery fails, the complaint workflow continues normally without interrupting complaint processing. Errors are recorded in system logs for future investigation.

This implementation ensures that communication remains reliable, transparent, and accessible while preventing email failures from affecting core complaint management operations.

#### **4.7 Attachment Implementation**

Students can attach files while submitting complaints. Files are uploaded to the Supabase Storage complaint-attachments bucket. Metadata such as file path, public URL, file type, and complaint ID is saved in the `complaint_attachments` table. This keeps file storage separate from complaint records while maintaining traceability.

#### **4.8 Analytics**

Analytics functionality was implemented to provide administrators with meaningful insights into complaint handling performance. Data-driven reporting assists decision-makers in evaluating service quality, workload distribution, and complaint trends across departments.

The analytics module aggregates complaint information from the database and generates statistical summaries based on status, category, priority level, department, and resolution performance. These statistics are displayed through dashboard cards and graphical visualizations.

The system includes custom SVG-based trend charts that display complaint activity over time. These visualizations help administrators identify complaint patterns, workload fluctuations, and potential operational issues requiring attention.

## **4.9 Botpress Chatbot Integration**

The Student Issue Tracker integrates Botpress Cloud Webchat to provide automated assistance and improve user experience. The chatbot serves as an interactive support tool that helps students understand system functionality and complaint procedures.

The chatbot is implemented as a floating widget positioned within the Student Dashboard. Users can interact with the chatbot to obtain information regarding complaint categories, priority selection, anonymous reporting options, complaint tracking, assignment procedures, notifications, ratings, and feedback processes.

To improve usability, the chatbot widget supports drag-and-drop functionality. Users can reposition the widget according to personal preferences and screen layout requirements. The selected position is stored locally, ensuring a consistent experience across future sessions.

Botpress APIs also support advanced integrations such as complaint submission assistance and complaint status retrieval when configured with appropriate authentication credentials. These capabilities create opportunities for future chatbot-driven workflow automation.

The chatbot significantly improves accessibility and reduces the need for manual support by providing immediate responses to frequently asked questions.

## **4.10 Voice Assistant, Search, and Responsive UI Implementation**

To improve accessibility and user engagement, the Student Issue Tracker includes a browser-based voice assistant implemented using Speech Synthesis APIs. The voice assistant announces important complaint events, notification updates, and workflow changes, helping users remain informed about system activities.

The assistant is displayed as a draggable floating widget that users can move anywhere on the screen. This flexibility prevents interference with dashboard content and enhances usability across different screen sizes.

The system also includes advanced search functionality designed to simplify complaint management. Users can search complaint records using complaint title, description, complaint ID,

category, priority, status, department, route information, student information, and assigned faculty member details.

Search operations are optimized to support quick retrieval of complaint information, enabling users to locate records efficiently even when complaint volumes increase.

Responsive design principles were applied throughout the application to ensure compatibility across mobile phones, tablets, laptops, and desktop computers. Responsive layouts, flexible components, adaptive menus, and optimized modal windows provide a consistent user experience regardless of device type.

Security measures complement these features through Supabase authentication, protected API routes, role validation mechanisms, Row Level Security policies, password validation, environment variable protection, and secure session management. Together, these controls help protect sensitive complaint information from unauthorized access.

**Table 4.3: Complaint Lifecycle Statuses**

| Status      | Meaning                                                                     |
|-------------|-----------------------------------------------------------------------------|
| Submitted   | Student submits complaint; system routes it to HOD/DSA/Supervisor.          |
| Rejected    | HOD or DSA rejects invalid or unclear complaint.                            |
| In Progress | Complaint is assigned to Faculty Member.                                    |
| Resolved    | Faculty Member marks assigned complaint as resolved.                        |
| Closed      | DSA or Supervisor closes resolved complaint.                                |
| Escalated   | Reserved status or flag for overdue or attention-needed complaint tracking. |

**Table 4.4: Implemented API Endpoint Summary**

| Endpoint                         | Purpose                                                                        |
|----------------------------------|--------------------------------------------------------------------------------|
| /api/auth/profile                | Loads authenticated profile.                                                   |
| /api/complaints                  | Creates and lists complaints according to user scope, role and category route. |
| /api/complaints/suggest          | Returns category, priority and similarity suggestions.                         |
| /api/complaints/[id]/review      | HOD/DSA accept or reject action.                                               |
| /api/complaints/[id]/assign      | Assigns complaint to Faculty Member.                                           |
| /api/complaints/[id]/status      | Faculty marks assigned complaint resolved.                                     |
| /api/complaints/[id]/finalize    | DSA/Supervisor closes resolved complaint.                                      |
| /api/complaints/[id]/rate        | Student saves rating and feedback.                                             |
| /api/complaints/[id]/comments    | Adds complaint comments.                                                       |
| /api/complaints/[id]/attachments | Stores attachment metadata.                                                    |
| /api/complaints/analytics        | Returns status, category, priority, department and trend chart data.           |
| /api/complaints/weekly-report    | Returns weekly report summary.                                                 |
| /api/departments                 | Department management.                                                         |
| /api/users                       | Staff/faculty account creation.                                                |
| /api/bot/submit                  | Botpress complaint submission integration.                                     |
| /api/bot/status                  | Botpress complaint status integration.                                         |

# **CHAPTER 5**

## **USER GUIDE**

This chapter provides a role-based guide for using the Student Issue Tracker. Actual interface screenshots can be inserted in place of the placeholders. Each placeholder is labeled with the expected screenshot title so that the final UI images can be added easily during final formatting. The guide reflects the current interface, including the Issue Tracker welcome page, circular logo, role dashboards, modal complaint details, search, profile menu, chatbot, and draggable assistant widgets.

## 5.1 Getting Started

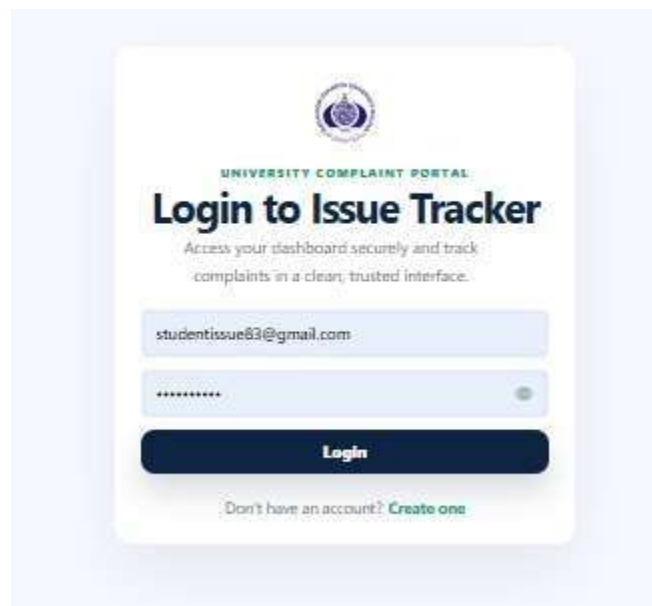
Users access the system through the web application. The welcome page displays the Issue Tracker title with the BZU Complaint System subtitle and provides Login and Register actions. After authentication, the dashboard adapts according to the role stored in the user profile



**Figure 5.1: Welcome Page**

## 5.2 Login Page

The user enters their email and password on the Login page. After successful authentication, Supabase returns a session and the application loads the user profile. The role stored in the profile determines the dashboard content displayed to the user. The profile picture menu provides Edit Profile and Logout actions and is optimized for mobile display.



**Figure 5.2: Login Page**

### 5.3 Registration Page

New students can register by providing their name, email, password, and department on the Registration page. Password rules are validated by the system to ensure account security. Staff accounts are created by authorized HOD, DSA, or Supervisor users through the system management interface, not through the public registration form.



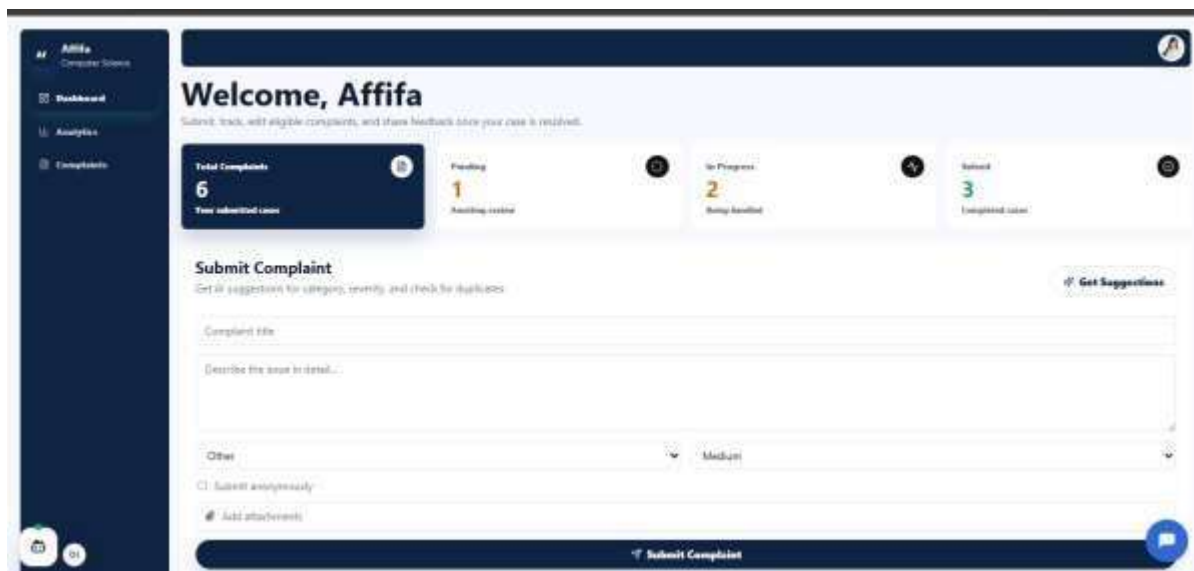
**Figure 5.3: Registration Page**



## 5.4 Student Dashboard

The Student Dashboard allows complaint submission, complaint tracking, searchable complaint records, modal-based complaint viewing, notifications through the Analytics section, chatbot support, and feedback submission. Students can view all submitted complaints and their current statuses without contacting staff manually.

Students enter title, description, category, priority, anonymous option, and attachments in the complaint submission form. The form validates required fields and sends data to protected API routes. Final routing is determined by the category: Academic and Behavior-related complaints go to HOD; Administrative, Facilities, and Other complaints go to DSA.



**Figure 5.4: Student Dashboard with Complaint Submission Form**

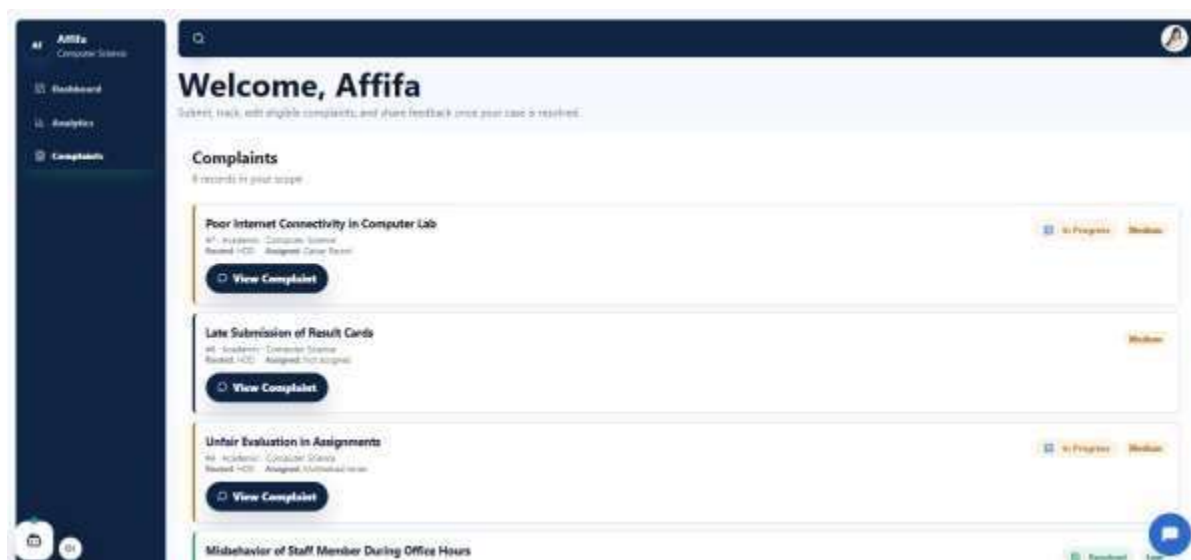
## 5.5 AI Suggestions Panel

After entering a complaint description, the AI Suggestions Panel displays the predicted category, suggested priority, and a possible similarity score comparing the new complaint with existing records. Students can accept the suggestion or manually adjust the category and priority before final submission.

**Figure 5.5: AI Suggestions Panel**

## 5.6 Complaint Tracking Interface

After submission, students can search and track complaint statuses such as Submitted, In Progress, Resolved, Closed, or Rejected through the complaint tracking interface. Complaint descriptions are not shown directly on cards; students use the View Complaint button to open the complete detail popup. This improves transparency while keeping the dashboard concise.

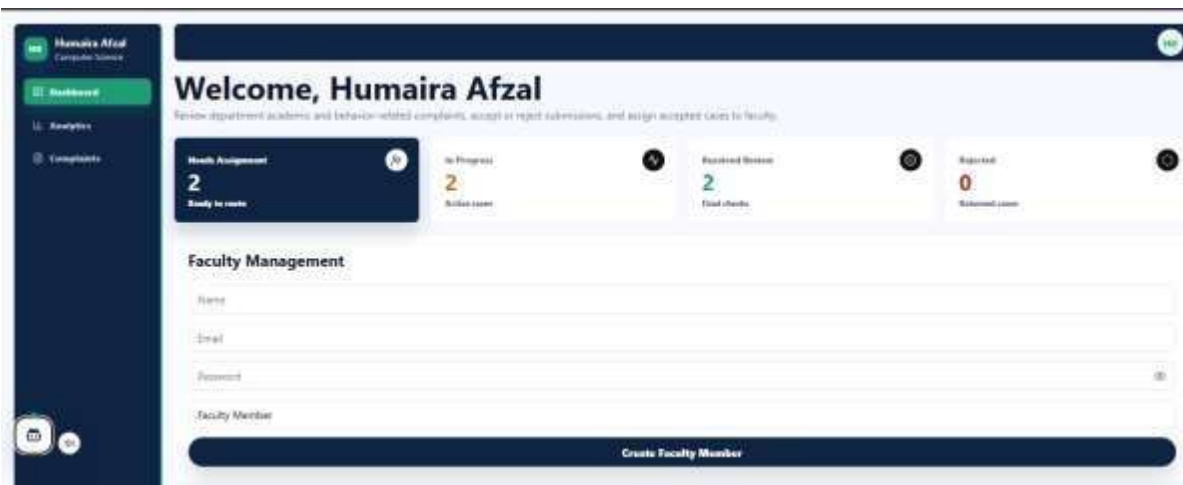


**Figure 5.6: Complaint Tracking Page**

## 5.7 HOD Dashboard

The HOD Dashboard is designed to manage Academic and Behavior-related complaints routed to the Head of Department. After authentication, HOD users can access complaints associated with their department through a centralized interface. The dashboard allows HODs to review

complaints, assign Faculty Members, add and manage departmental faculty accounts, and monitor complaint progress. Complaint information is displayed using organized cards and status indicators, enabling quick identification of pending, assigned, and resolved complaints.



**Figure 5.7: HOD Dashboard**

### 5.7.1 HOD Analytics Dashboard

The HOD Dashboard analytics section provides visual insights into departmental complaint handling through graphs. It shows key metrics such as total complaints, resolved, pending, and in-progress cases, along with category distribution and resolution rates. These visualizations help the HOD quickly understand trends and evaluate overall departmental performance.



**Figure 5.7.1: HOD Analytics Dashboard**

### 5.7.2 Complaint Review Modal

The Complaint Review Modal allows HOD users to view complete complaint details, including the description, category, priority, attachments, and student information when available. Based on the provided information, the HOD can accept or reject the complaint. Invalid, duplicate, unclear, or out-of-scope complaints may be rejected, while valid complaints proceed to the assignment stage.

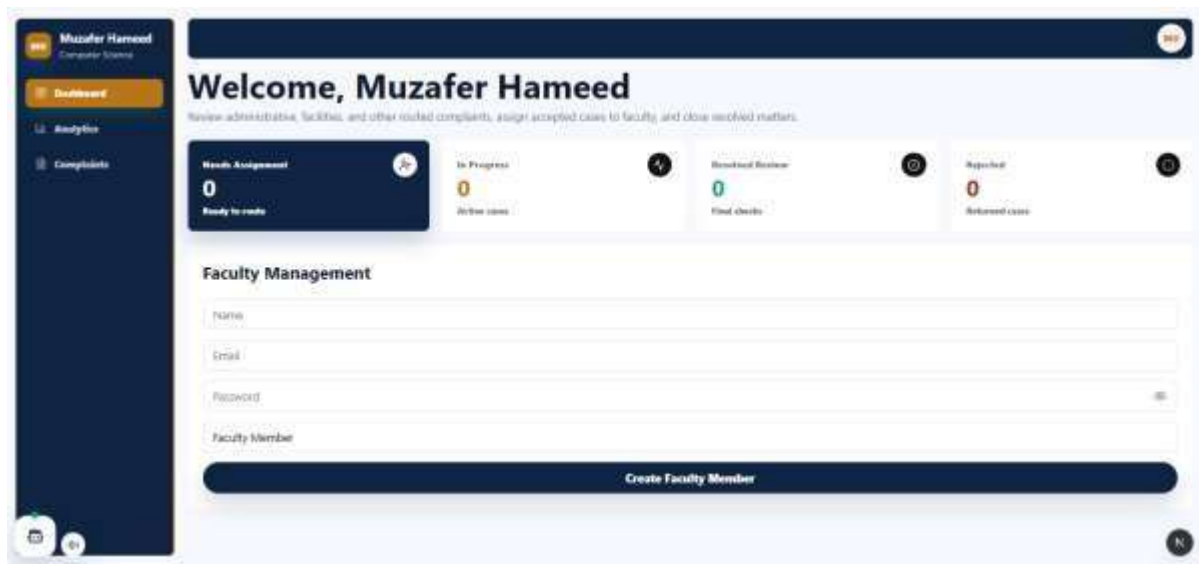


**Figure 5.7.2: Complaint Review Modal**

## 5.8 DSA Dashboard

The dashboard and analytics section provide an overview of complaint activities. In this system, Faculty can also be added similar to the HOD role, and analytics functions work in the same way for performance tracking and insights.

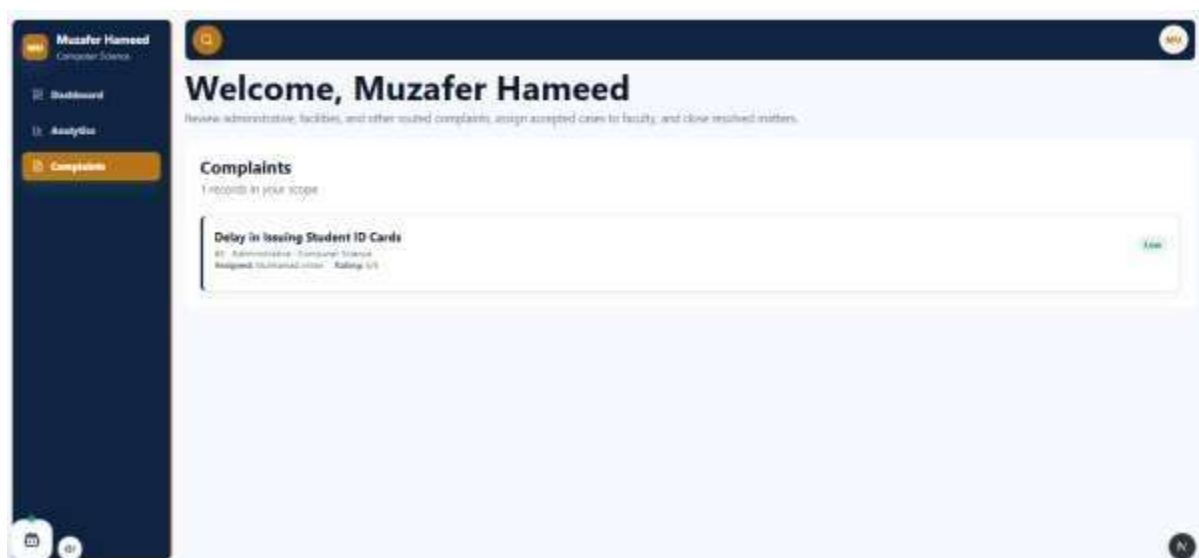
The DSA Dashboard focuses on routed complaints including Administrative, Facilities, and Other categories. These complaints are systematically managed to ensure proper assignment, monitoring, and resolution within the institution.



**Figure 5.8: DSA Dashboard**

### 5.8.1 Complaint Section

Through this section, DSA users can review detailed complaint information, verify submissions, and take necessary actions such as accepting or rejecting complaints based on their validity and relevance. They can also assign Faculty Members to handle specific cases and monitor the progress of each complaint until it is properly resolved or closed. This ensures that all non-academic and general institutional issues are handled in a structured and accountable manner.



**Figure 5.8.1: Complaint Section**

## 5.9 Faculty Member Dashboard

Faculty Members can view their assigned complaints on the dashboard, where each complaint is shown in a card format with limited details. The full complaint information is not visible directly; instead, the “Complaint Details” button is used to open the complete record.

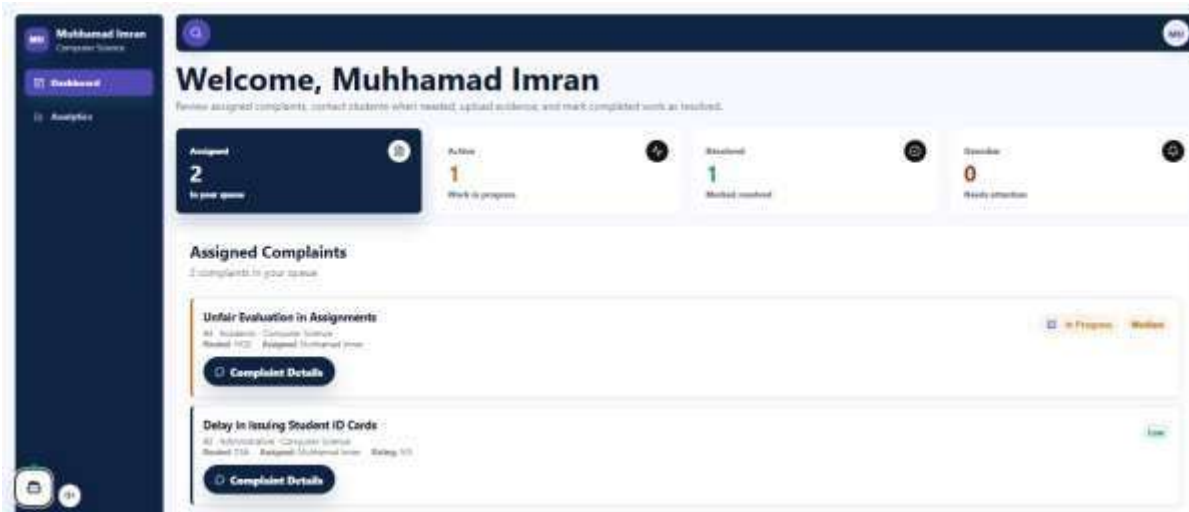


Figure 5.9: Faculty Member Dashboard

### 5.9.1 Complaint details

When the Complaint Details popup is opened, it displays all information including the complaint description, category, and the student who submitted it. If any attachments were added, they are also shown. The popup further includes a “Contact Student” button for direct communication when an email is available, along with a “Mark as Resolved” button to close the complaint after completing the required action.

The screenshot displays the 'Student Issue Tracker' interface. A modal window titled 'Complaint #4' is open, showing details for a complaint. The modal includes fields for Student (Alfa), Contact Email (studentissue83@gmail.com), Priority (Medium), Category (Academic), Department (Computer Science), and Status (In Progress). A description box contains the text: 'Many students have reported inconsistency in marking assignments, where similar answers are graded differently by instructors. This creates confusion and a sense of unfairness among students. It is requested that a standardized marking rubric be used to ensure transparency and fairness in academic evaluation.' Below the description is an 'Attachments' section with a placeholder for an image. At the bottom of the modal are buttons for 'Contact Student', 'Upload solved evidence', and a green 'Mark Solved' button. The background shows a sidebar with 'Welcome, M...' and 'Assigned 2 in your queue', and a main area with 'Assigned Complaints' and two complaint cards: 'Unfair Evaluation in Assignment' and 'Delay in Issuing Student ID Card'.

Figure 5.9.1 complaint details

## 5.10 Supervisor Dashboard

The Supervisor Dashboard provides complete visibility across all departments. Supervisors can view all complaint records, manage departments and staff accounts, monitor activity logs, review analytics. The Supervisor has the authority to add departments along with their respective HODs and DSA users in the system. If any HOD or DSA is changed later, the Supervisor can update and replace them accordingly.

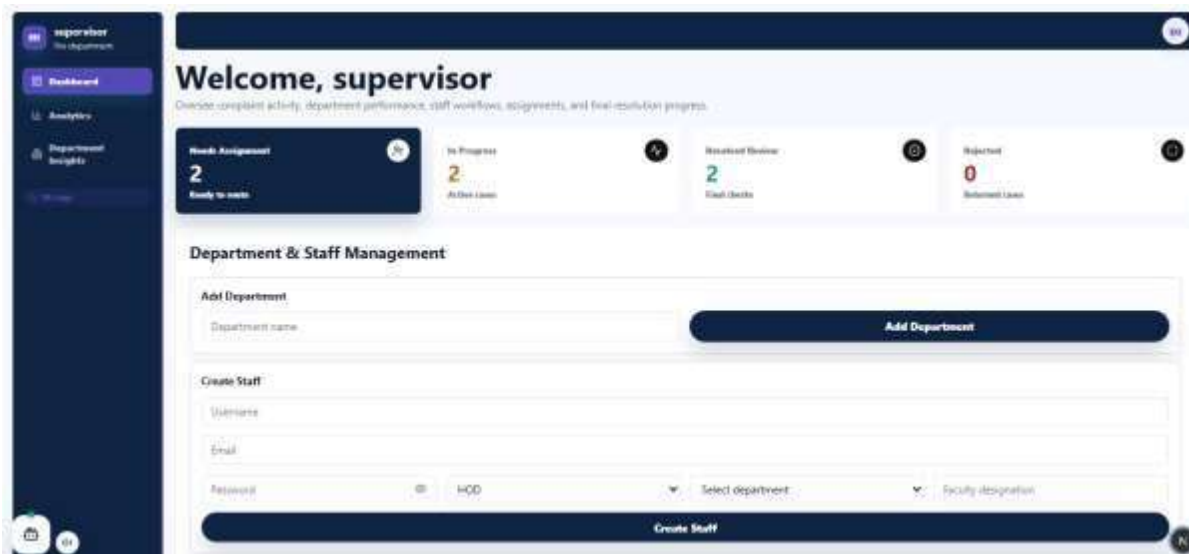


Figure 5.10: Supervisor Dashboard

### 5.10.1 Analytic section

The analytics section allows users to view the overall trend of complaints over time. It shows how many complaints are solved, in progress, or closed, helping to identify which status is most common at a given time.

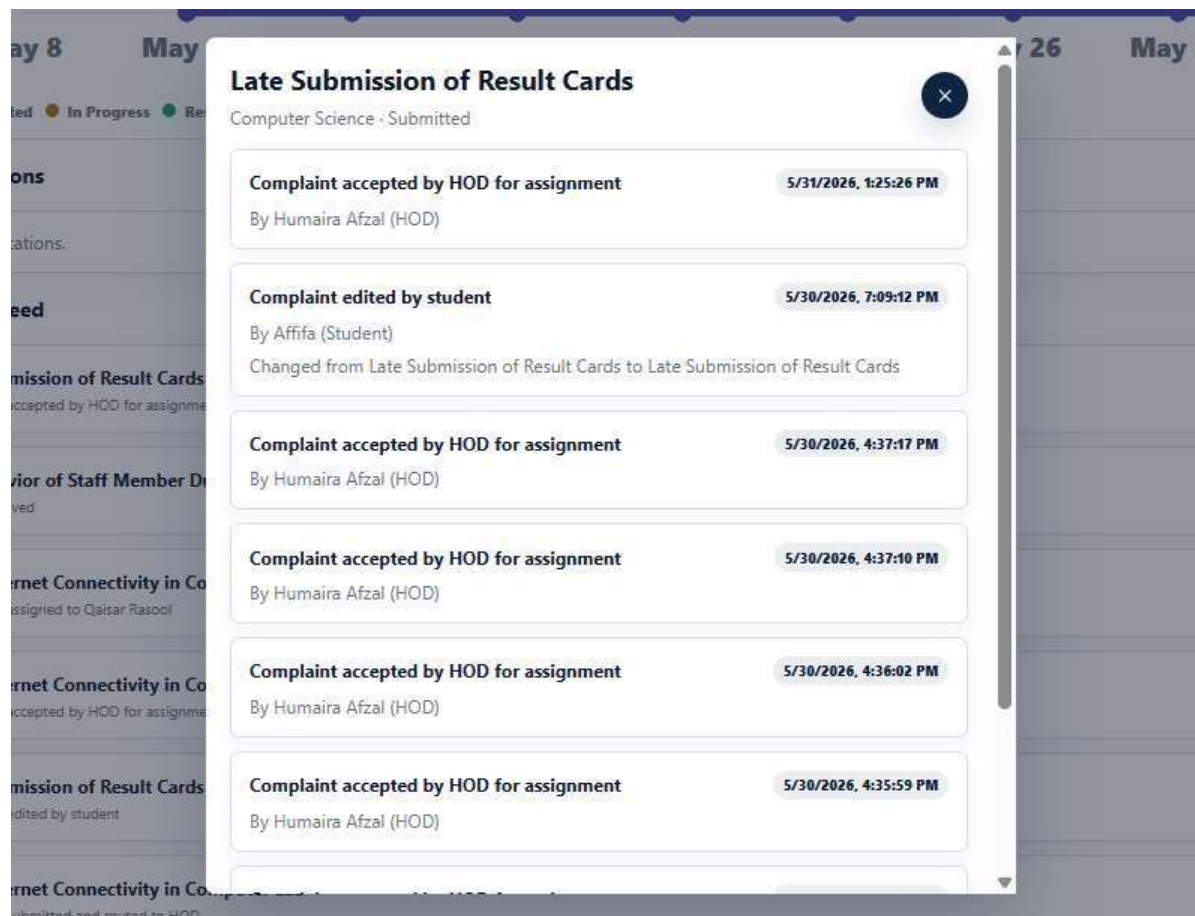


Figure 5.10.1 Analytic section



### 5.10.2 Activity Feed

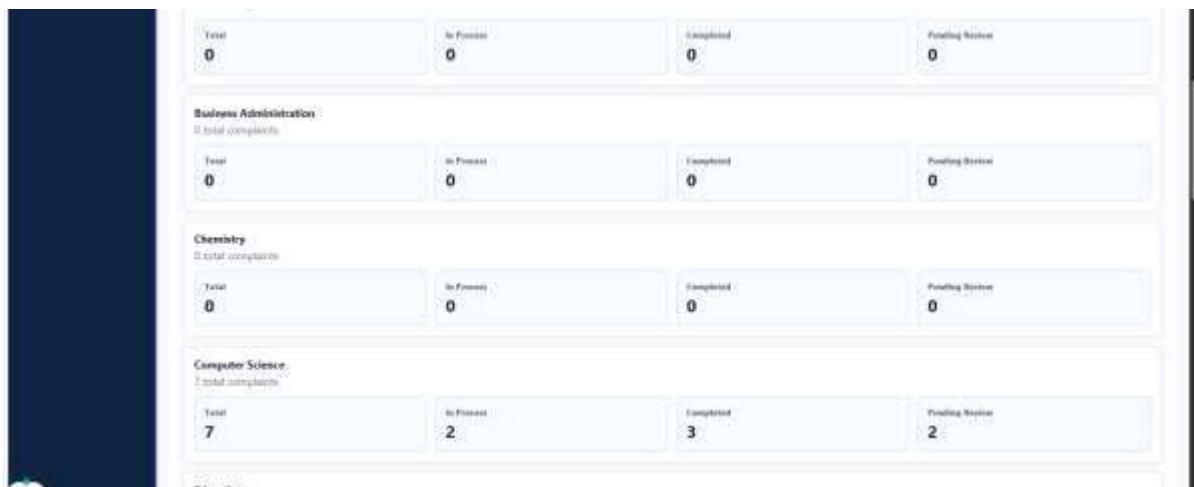
In the activity feed, when a complaint is clicked, a popup opens showing full details of the complaint, including who submitted it, who accepted it, and who resolved it with time and date . It provides a complete activity history of the complaint in one view.



**Figure 5.10.2 Activity Feed**

### 5.10.3 Departmental Insight

The department insight section provides a detailed overview of each department's performance by showing the total number of complaints . It breaks down how many complaints are in progress, completed, or still pending within each department. This helps in comparing departmental efficiency and overall workload management.



**Figure 5.10.3 Departmental Insight**

### 5.11 Notifications Interface

Notifications inform users about complaint submission, review, assignment, resolution, rejection, closure, and feedback events. Notification data is available through dashboard sections and optional email alerts. The notifications are presented within the Analytics section to reduce interface clutter while ensuring users can easily access important updates.

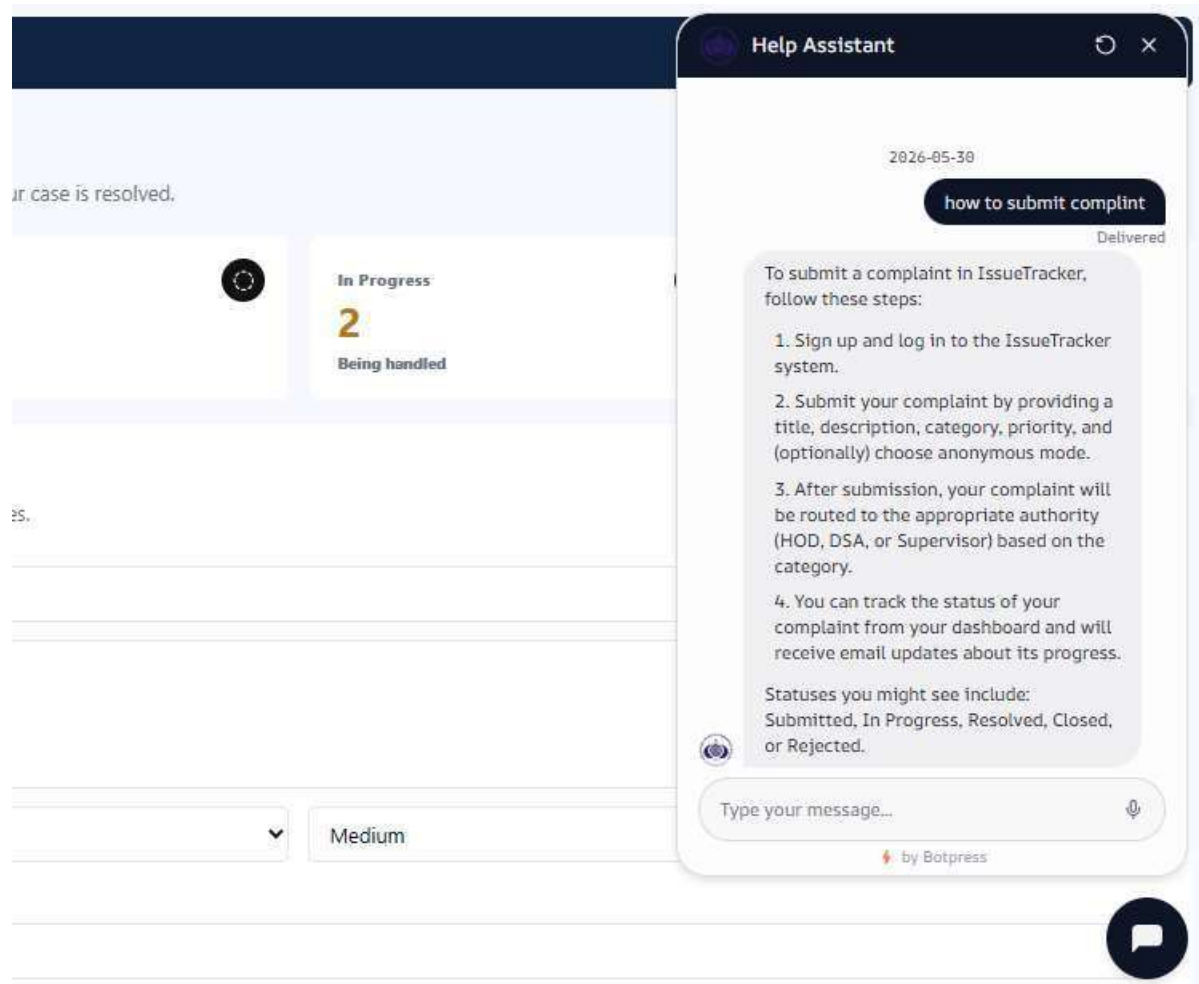


**Figure 5.11: Notifications**

### 5.12 Botpress Chatbot

The Botpress chatbot helps students understand complaint categories, priority selection, submission process, status tracking, faculty assignment, email alerts, rating, and feedback procedure. The chatbot floating icon is draggable so students can move it away from important

screen content. Students can interact with the chatbot to obtain information and procedural guidance at any time.



**Figure 5.12: Botpress Chatbot**

### 5.13 Common Status Meanings

| Status      | User Meaning                                                          |
|-------------|-----------------------------------------------------------------------|
| Submitted   | Complaint has been submitted and is waiting for review or assignment. |
| In Progress | Complaint has been assigned to a Faculty Member.                      |
| Resolved    | Faculty Member has completed the assigned work.                       |

|           |                                                                                   |
|-----------|-----------------------------------------------------------------------------------|
| Closed    | DSA or Supervisor has finalized the complaint.                                    |
| Rejected  | Complaint was rejected during review.                                             |
| Escalated | Complaint is marked as needing attention due to delay or future escalation rules. |

**Table 5.1: User-Facing Complaint Status Guide**

# **CHAPTER 6**

# **CONCLUSION**

## 6.1 Testing Strategy

Testing was performed around complete role-based workflows. Each user role was tested separately and then tested as part of the complete complaint lifecycle. The purpose was to verify authentication, permissions, data saving, fixed category routing, notifications, assignment, resolution, closure, rating, feedback, search, modal detail views, responsive profile dropdown, draggable assistants, and analytics.

| Test ID | Scenario                   | Expected Result                                                                  | Status |
|---------|----------------------------|----------------------------------------------------------------------------------|--------|
| TC-01   | Student registration/login | Valid account logs in and opens Student dashboard.                               | Pass   |
| TC-02   | Complaint submission       | Complaint saved with Submitted status.                                           | Pass   |
| TC-03   | AI suggestion              | Category and priority suggestions returned.                                      | Pass   |
| TC-04   | Attachment upload          | File saved in Supabase Storage and metadata saved.                               | Pass   |
| TC-05   | HOD/DSA review             | Complaint accepted or rejected according to category route and department scope. | Pass   |
| TC-06   | Faculty assignment         | Complaint changes to In Progress and faculty is notified.                        | Pass   |
| TC-07   | Faculty resolution         | Assigned faculty marks complaint Resolved.                                       | Pass   |
| TC-08   | Finalization               | DSA/Supervisor closes resolved complaint.                                        | Pass   |
| TC-09   | Rating                     | Student submits rating and feedback once.                                        | Pass   |
| TC-10   | Analytics and search       | Dashboard counts and complaint search match scoped complaint records.            | Pass   |
| TC-11   | Role restriction           | Unauthorized users cannot access another role workflow or contact action.        | Pass   |
| TC-12   | Email fail-safe            | Complaint workflow continues if email sending fails.                             | Pass   |

**Table 6.1: Functional Test Cases**

## 6.2 Results

The implemented system successfully supports role-based complaint management. Students can submit, search, view, and track complaints; HOD and DSA users can review and assign routed complaints; Faculty Members can view detailed assignments, contact students where appropriate,

upload evidence, and resolve work; and Supervisors can manage departments, staff, analytics, finalization, and activity logs. Notifications, email alerts, chatbot guidance, voice-assistant messages, modal complaint details, and responsive dashboards improve communication between stakeholders.

### **6.3 Conclusion**

The Student Issue Tracker provides a centralized, secure, and role-aware complaint management solution for university departments. It improves record keeping, transparency, assignment control, accountability, reporting, and user experience. The use of Next.js, Supabase, role-based API checks, AI utility functions, chatbot guidance, searchable complaint queues, and responsive dashboards makes the system practical for academic deployment and easy to extend in future versions.

### **6.4 Future Work**

Future enhancements include native Android and iOS mobile applications, deep learning based NLP models for improved complaint classification, semantic duplicate detection using embeddings, real-time WebSocket notifications, automated escalation scheduling for overdue complaints, downloadable PDF weekly and monthly reports, advanced filtering and export options, multi-campus deployment, richer staff workload dashboards, in-app notification read/unread controls, and configurable routing management through an administrative interface.

## REFERENCES

Next.js Documentation. <https://nextjs.org/docs>

React Documentation. <https://react.dev/>

Supabase Documentation. <https://supabase.com/docs>

Supabase Auth Documentation. <https://supabase.com/docs/guides/auth>

Supabase Storage Documentation. <https://supabase.com/docs/guides/storage>

PostgreSQL Documentation. <https://www.postgresql.org/docs/>

Lucide React Documentation. <https://lucide.dev/>

Gmail SMTP / Google App Password Documentation.

<https://support.google.com/accounts/answer/185833>

Botpress Documentation. <https://botpress.com/docs>

Pressman, R. S. Software Engineering: A Practitioner's Approach.

Sommerville, I. Software Engineering.

Larman, C. Applying UML and Patterns.

Elmasri, R., & Navathe, S. Fundamentals of Database Systems.



## APPENDICES

### APPENDIX A: DATABASE TABLE SUMMARY

PostgreSQL enums support user roles, complaint statuses, complaint categories, complaint priorities, and routed roles. Foreign keys connect profiles, departments, complaints, comments, attachments, notifications, category routes, and activity logs. The current workflow uses fixed category routing in application logic while the database schema also stores route rules for future configurability. This structure supports normalized storage and prevents unnecessary duplication.

### APPENDIX B: DETAILED WORKFLOW RULES

- Academic complaints route to HOD by default.
- Behavior-related complaints route to HOD by default.
- Administrative complaints route to DSA by default.
- Facilities complaints route to DSA by default.
- Other complaints route to DSA by default.
- Faculty Members can only update complaints assigned to them.
- Students can only rate complaints that belong to them and are Resolved or Closed.

Students can only rate complaints that belong to them and are Resolved or Closed; eligible submitted complaints can be edited only once.

- DFD Level 0 focuses on high-level interaction between users, system, email service, chatbot, voice assistant and Supabase.
- DFD Level 1 decomposes the system into authentication, complaint submission, AI suggestion, routing, review, assignment, resolution, finalization, feedback, notifications, analytics, search, profile management, chatbot support, voice assistant, and management.
- Use Case Diagram identifies the complete permissions and responsibilities of each actor.
- Sequence Diagrams explain chronological message flow for login, submission, assignment, resolution, closure and feedback.
- ERD explains data relationships and foreign key dependencies.